

Table of Contents

1	Rezumat.....	3
1.1	Stadiul Actual	3
1.2	Fundamentare teoretică	3
1.2.1	ESP32	3
1.2.2	Arduino Uno.....	4
1.2.3	ESP32-CAM.....	4
1.2.4	Comunicarea prin Wi-Fi.....	5
1.2.5	Protocolul HTTP în sisteme IoT.....	5
1.3	Implementare	5
1.4	Rezultate experimentale	7
2	Activity planning	8
3	State of the Art.....	9
3.1	Promise vs. Reality of Energy Efficiency	10
3.2	Smart Home social impacts.....	11
3.3	Existing commercial products.....	11
3.3.1	Alexa	11
3.3.2	Google Nest.....	11
3.3.3	Apple HomePod	12
3.4	Conclusion	12
4	Theoretical fundamentals	13
4.1	ESP32	14
4.2	ESP32-CAM	15
4.3	Arduino Uno	15
4.4	Temperature sensor, DHT11	16
4.5	Fingerprint sensor module	16
4.6	DC Motor	16
4.7	Solenoid lock.....	17
4.8	Magnetic sensor.....	18
4.9	Ultrasonic sensor	18
4.10	Relay module	19
4.11	Motor driver	19
4.12	DC Fan	20
4.13	Wireless communication in Smart Home Systems.....	20
4.14	Wi-Fi as a means of communication.....	21
4.15	HTTP Protocol in IoT systems.....	21

4.16	Network Architecture and Communication Protocol.....	21
4.17	Potential Threats and Vulnerabilities	22
5	Implementation.....	24
5.1	System Architecture and Block Diagram	24
5.2	Flashing the ESP32-CAM	25
5.3	Integrating the fingerprint module.....	27
5.3.1	Code explanation	27
5.4	Integration of the main controller, ESP32 30-pin	29
5.4.1	Code explanation	29
5.5	Integration of the web-hosting controller, ESP32-CAM	32
5.5.1	Code explanation	32
5.6	Hardware Implementation.....	36
5.6.1	Methodology and Limitations.....	36
5.6.2	Hardware Implementation – Overview	36
5.6.3	Physical Implementation – Wiring Schemes	37
6	Experimental Results	39
6.1	Window Monitoring.....	39
6.2	Front Door Monitoring.....	40
6.3	Interior Lighting Control	41
6.4	Garage Door Control and Status	42
6.5	Temperature Control.....	43
7	Conclusion	45
8	References	46
9	Annexes.....	50
10	Curriculum Vitae	77

1 Rezumat

1.1 Stadiul Actual

Automatizarea locuințelor a avut parte de o expansiune majoră în ultimii ani, contribuind la îmbunătățirea calității vieții prin progresul rapid al tehnologiei. Un avantaj major al acestor sisteme este accesibilitatea lor, fiind ușor de controlat printr-un smartphone, laptop sau chiar comenzi vocale. Această automatizare reprezintă un exemplu al sistemelor „Internet of Things” (IoT), care permit controlul și monitorizarea de la distanță și a dispozitivelor prin internet (Figura 1). Aceste sisteme oferă multiple beneficii, inclusiv un nivel de securitate sporită datorită măsurilor avansate de protecție și supravegherii continue, un confort ridicat prin ajustări rapide și economii financiare mulțumită eficienței energetice [1].

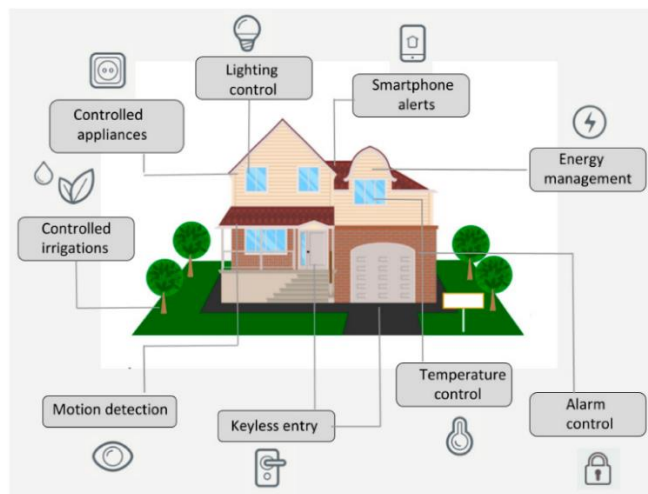


Figura 1 Locuință smart bazată pe sisteme IoT

1.2 Fundamentare teoretică

În ultimii ani, conceptul de locuință smart a evoluat de la o noțiune futuristă la o componentă aproape indispensabilă pentru numeroase case și familii. Casele smart integrează tehnologii avansate precum senzori, microcontrolere, protocoale de comunicare wireless și sisteme automatizate, pentru a oferi un confort sporit, securitate și eficiență energetică. Acest progres tehnologic a fost accelerat de cererea crescândă a consumatorilor pentru gestionarea facilă și eficientă a sistemelor din casă.

Statisticile arată că, doar anul trecut, în Statele Unite, un număr de aproximativ 70 de milioane de locuințe utilizau cel puțin un dispozitiv pentru case inteligente. Acest număr indică o creștere de aproape 10% față de anul precedent, evidențiind ascensiunea remarcabilă a popularității dispozitivelor pentru casele smart [38].

1.2.1 ESP32

Plăcuța de dezvoltare ESP32, ilustrată în Figura 2, este un dispozitiv integrat dezvoltat de Espressif Systems, apreciat pentru versatilitatea sa în aplicații de tip IoT și embedded. Acesta dispune de un procesor dual-core Tensilica LX6, conectivitate Wi-Fi și Bluetooth, dar și o gama variată de interfețe, precum ADC, DAC și controlere PWM [6]. Capacitățile de comunicare

wireless și interfețele hardware îl fac ideal pentru proiectele care necesită procesare în timp real, conectivitate și consum redus.



Figura 2 Plăcuța ESP32

1.2.2 Arduino Uno

Plăcuța de dezvoltare Arduino Uno, ilustrată în Figura 3, este bazată pe microcontrolerul ATmega328P și este destinată utilizării facile, în special în context educațional și pentru pasionați. Funcționează la o frecvență de 16 MHz, dispune de 32 KB memorie flash, 14 pini digitali I/O și 6 canale de intrare analogice [9]. Include un conector USB tip B pentru programare și comunicare serială, precum și reglare integrală a tensiunii pentru alimentare prin USB sau surse externe.



Figure 3 Plăcuța Arduino Uno [14]

1.2.3 ESP32-CAM

Modulul ESP32-CAM, ilustrat în Figura 4, este bazat pe microcontrolerul ESP32 și este conceput pentru captarea și transmitere wireless a imaginilor în sisteme embedded. Include un modul de cameră OV2642 cu rezoluție de până la 2MP, fiind ideal pentru aplicații cu consum redus și costuri scăzute, precum supravegherea sau procesarea imaginilor. Dispune de un slot pentru card microSD pentru stocarea datelor. Spre deosebire de alte plăci de dezvoltare, acesta nu are interfață USB nativă, necesitând un adaptor USB-serial extern sau un dispozitiv intermediar pentru programare [7], [8].



Figura 4 Modulul ESP32-CAM [13]

1.2.4 Comunicarea prin Wi-Fi

Tehnologia de comunicare wireless prin Wi-Fi este utilizată la scară largă în sistemele IoT datorită capacității sale de a răspunde cerințelor utilizatorilor, oferind rate înalte de transfer de date, fiabilitate și compatibilitate cu rețele domestice de internet. În cadrul unui proiect IoT, multiple dispozitive se conectează la aceeași rețea Wi-Fi, astfel facilitând transmiterea și recepționarea informațiilor.

Dispozitivele pot funcționa drept clienți, servere, sau rol hibrid, în funcție de scopul aplicației. Cu toate acestea, comunicația prin Wi-Fi în medii interioare prezintă și dezavantaje, printre care atenuarea semnalului cauzată de pereți sau mobilier, interferențe generate de alte dispozitive electronice și rețele, precum și latența, care poate afecta transmisia rapidă a datelor, esențială pentru funcționarea eficientă a acestui tip de system.

1.2.5 Protocolul HTTP în sisteme IoT

În sistemele IoT, comunicația între dispozitive se bazează adesea pe protocoale standard precum HTTP, care reprezintă protocolul fundamental al comunicațiilor web. Protocolul HTTP facilitează schimbul de mesaje pentru a iniția acțiuni sau a actualiza stări. Dintre metodele HTTP utilizate cel mai mult în sistemele IoT, se remarcă cel mai mult următoarele: GET, PUT și POST [36].

Metoda GET este utilizată pentru a extrage date de pe un server, obținând informații precum conținutul unei pagini web sau starea unui senzor [36].

Metoda PUT permite actualizarea sau înlocuirea unei resurse la o adresă URL specifică, fiind idempotentă, ceea ce înseamnă că mai multe cereri PUT identice produc același rezultat [36].

În schimb, metoda POST este folosită pentru a transmite date către un server în vederea creării sau actualizării resurselor, fiind frecvent utilizată în proiecte IoT pentru a trimite date citite de la senzori, date de autentificare sau comenzi ale utilizatorului. Spre deosebire de PUT, POST nu este idempotentă, astfel că cereri repetate și identice pot genera rezultate diferite [36].

1.3 Implementare

Arhitectura sistemului de locuință smart adoptat (Figura 5) este structurată în jurul a trei componente principale, menționate anterior: modulul ESP32-CAM, microcontrolerul ESP32 și placa Arduino Uno. ESP32-CAM servește pe post de “host” al pentru interfața web a utilizatorului și modul de camera de securitate, în timp ce ESP32 gestionează senzorii și actuatoarele.

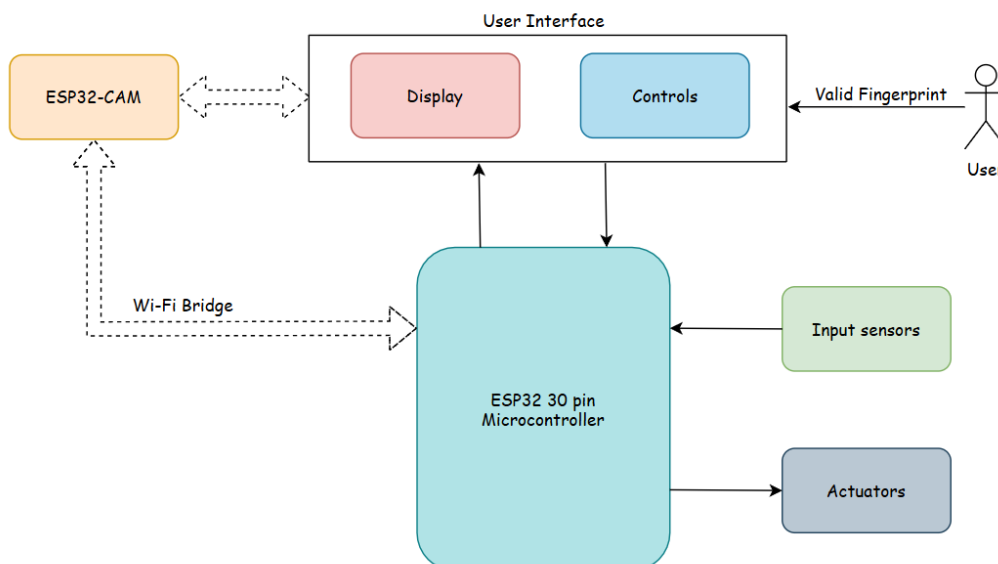


Figure 5 Diagrama bloc a soluției adoptate

Comunicarea între plăci se realizează exclusiv prin conexiune wireless, utilizând “bridge” Wi-Fi. Interfața web oferă un flux video în timp real de la camera integrată și date actualizate de la senzorii conectați, permițând totodată transmiterea comenzilor către ESP32 prin Wi-Fi. La rândul său, ESP32 coordonează senzorii și actuatorile, executând comenzile primite și transmițând actualizări despre starea senzorilor către ESP32-CAM pentru a asigura o funcționare sincronizată a sistemului. Interfața web a utilizatorului, care poate fi accesată dintr-un browser web este securizată prin autentificare biometrică bazată pe amprentă. După introducerea unei amprente valide în senzorul de amprentă, interfața afișează starea sistemului, inclusiv informații despre starea de deschidere a ușii și a ferestrei, ușa garajului, temperatura ambientală și condiții de iluminare, oferind utilizatorului posibilitatea de a controla ușa garajului, de a încuia sau descuia ușa principală, a seta limite de temperatură pentru un sistem de tip termostat sau de a controla intensitatea luminoasă din casă.

Aplicația web, ilustrată în Figura 6, prezintă o arhitectură adaptivă, fiind optimizată pentru compatibilitate și funcționalitate eficientă atât pe dispozitive mobile, cât și pe sisteme de tip desktop, asigurând o experiență de utilizare plăcută și intuitivă indiferent de tipul de ecran utilizat.

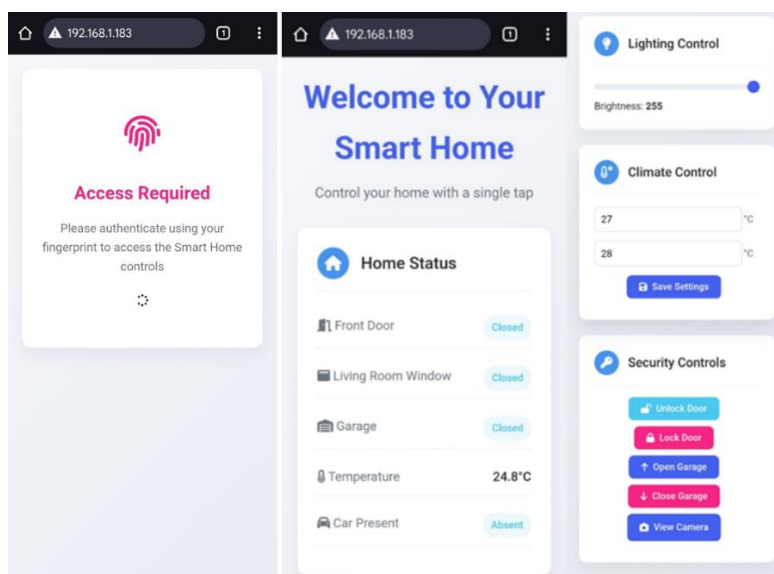


Figure 6 Aplicația web blocată și deblocată, vedere de pe mobil

1.4 Rezultate experimentale

Pentru testarea funcționalității sistemului, au fost efectuate mai multe experimente asupra acestuia pentru a determina capacitățile și comportamentul sistemului în diverse situații, evaluând eficiența și rapiditatea acestuia. Testele au inclus scenarii precum ajustarea intensității iluminatului interior prin intermediul unui cursor de tip slider în aplicația web, deschiderea ferestrei și a ușii cu verificare a stărilor ulterioare (Figura 7, 8), stabilirea pragurilor de temperatură pentru subsistemul de tip termostat, deschiderea ușii garajului alături de funcția de confort de închidere automată și alte acțiuni relevante.

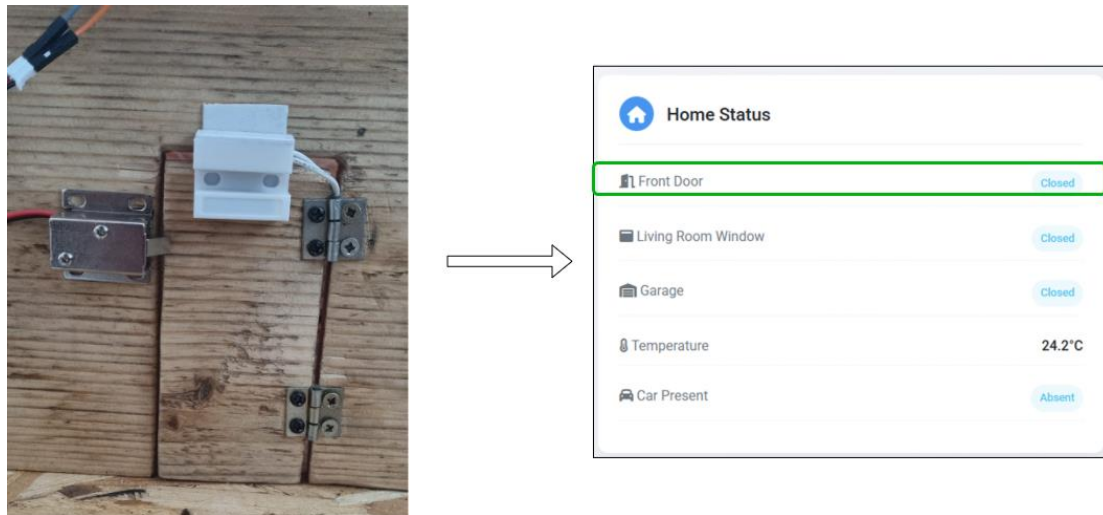


Figura 7 Starea ușii în aplicația web când este închisă

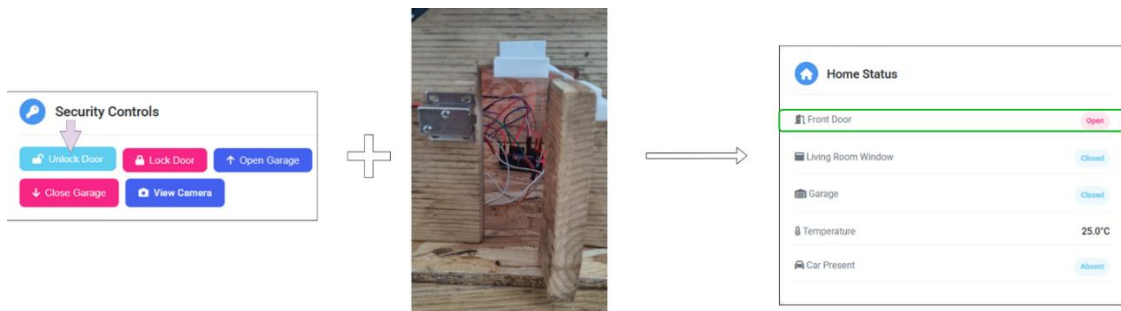


Figura 8 Starea ușii în aplicația web după deblocarea și deschiderea acesteia

Starea ușii de la intrare este monitorizată prin intermediul unui senzor magnetic, similar celui utilizat pentru monitorizarea stării ferestrei, ambele fiind gestionate de microcontrolerul ESP32. La închiderea ușii sau ferestrei, senzorul magnetic indică starea închis, iar la deschidere, detectează starea deschis, transmițând în timp real schimbările către aplicația web. Spre deosebire de fereastră, ușa include un mecanism de blocare care permite deschiderea acesteia doar după deblocare, acțiune realizată prin butonul “Unlock Door” din secțiunea “Security Controls” a aplicației web. Din motive de securitate, la inițializarea sistemului, încuietoarea cu solenoid este setată în poziția blocat.

Testele efectuate asupra funcționalităților legate de ușa de garaj, setarea temperaturii și controlul iluminatului interior au demonstrat performanțe care se ridică la nivelul așteptărilor.

2 Activity planning

Task	Time Period	oct	nov	dec	jan	feb	mar	apr	may	jun
Choosing the topic	01.10.2024 08.10.2024	■								
Research the topic extensively	09.10.2024 27.11.2024	■	■							
Conducting a component requirements analysis	28.11.2024 19.12.2024		■	■						
Testing all the components individually	20.12.2024 02.02.2025			■	■					
Implementing the hardware	03.02.2025 13.03.2025					■	■			
Implementing the software	14.03.2025 24.04.2025						■	■		
Testing the functionalities	25.04.2025 16.05.2025								■	
Fixing bugs and optimization	16.05.2025 29.05.2025									■
Elaborating the documentation	30.05.2025 30.06.2025									■

3 State of the Art

Home automation has grown more and more over the last couple of years, as the quality of life of the masses is being positively affected by the almost out of hand evolution of technology. One massive advantage of these home automations is their ease of use, as they can be controlled via smartphone, laptop or even voice, with the help of voice assistants. Home automation is just an archetype of the systems that are called Internet of Things (IoT) Systems (Figure 9), which aim to remotely control and monitor devices by the way of the Internet. With these Home Automation Systems (HAS), the masses are looking at a series of benefits, such as better home security due to increased protection measures and constant monitoring, as well as an increased level of overall home comfort by having any adjustment at a few click away and reduced bills as energy management is one of the key features of these systems [1].

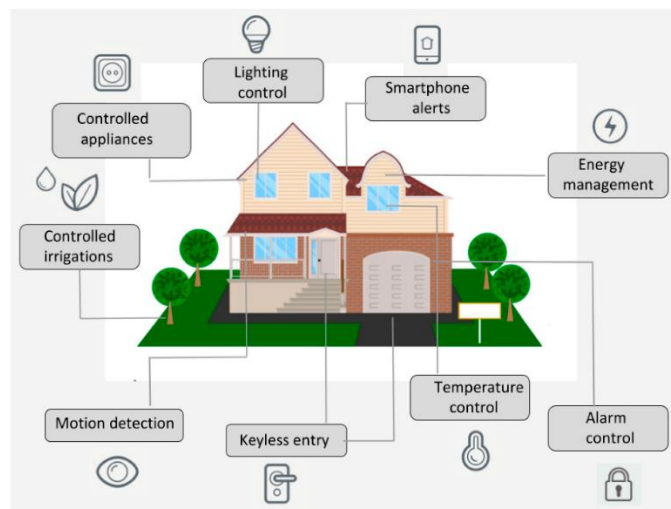


Figure 9 An IoT based smart home diagram [1]

Among the many challenges faced in the smart home technology, privacy and security are the most sensible topics and often overlooked, as some companies have “taken advantage” of these concerns and stepped onto the market showcasing their expertise in privacy and secure systems. Householders may be skeptical about certain technologies and losing the privacy and protection of their home, from a digital point of view. These affirmations are supported by a survey done in Austria, which shows that although people tend to trust their suppliers, there were some worries about their privacy being violated and potential breaches of confidentiality without their knowledge [2].

Smart home technologies are split into 3 large categories, as proposed by De Silva et al. (2012) [3] and Bowes et al. (2012) [3], based on what they look to offer:

- The first type represents homes that offers assistance to the occupants by recognizing activity. These services that are provided are also divided into yet another three categories: homes that support elderly care, homes that aid in nurturing young children and homes that provide general health management [3].
- The second category focuses on capturing multimedia data, such as videos or images, documenting different aspects of the inhabitants’ lives. While this approach enhances monitoring capabilities and seems beneficial, as mentioned above in the previous paragraph, it raises several questions regarding privacy and security [3].
- The third category is often referred to as the “surveillance home” and it is designed to process environmental data in order to predict and notify occupants of a potential incoming natural disaster or any kind of security threat. Almost no project has

successfully involved all the services that such surveillance-oriented smart home is intended to provide [3].

Since the 1990s and 2000s, smart homes have re-emerged as key tools for both optimizing energy efficiency and the quality of life. Over time, numerous definitions have been proposed, going from those emphasizing integrated communication systems, to those highlighting advanced, user-centered technologies. Along with this evolving term of smart homes, new related terms have emerged, suggesting its broad and various nature, such as smart home services, household IoT, smart devices and home automation technologies. Below is represented the evolution of the term “smart home” over the years (Table 1) [4].

Date	Definition
1992	The unification of various household services through a centralized communication system enables efficient, secure, and comfortable home operation, while incorporating a high level of intelligent functionality and adaptability
2003	A technology enhanced home that utilizes computing and communication systems to recognize and adapt to the preference of its inhabitants, aiming to improve ease of living, safety, enjoyment and connectivity with the outside world.
2012	A domestic setting enhanced with contextual awareness and autonomous systems, capable of adjusting to the occupant’s behavior while offering diverse functional support.
2014	A responsive, bidirectional interaction framework that enables seamless communication between the home and its residents
2017	A smart home monitors and interprets data from the household environment, communicates information to its residents and providers and improves control over various home systems
2019	A home integrated with intelligent technologies designed to deliver personalized services to its occupants

Tabel 1 Evolution of the term Smart Home over the years [4]

3.1 Promise vs. Reality of Energy Efficiency

Government officials and energy sector leaders often promote a future where households actively monitor and manage their energy use with the help of intelligent energy systems and digital

communication tools. As part of this vision, one European Commission has set a goal that by 2030, 80% of residential energy should be manageable via digital technologies in 80% of homes[5].

Despite this ambition, there is a major lack of independent studies that thoroughly explore both the energy-related and social impacts of smart home solutions. Investigations often focus on individual devices, involve participants who are already engaged with energy-saving practices, or are designed to find ways to boost adoption rates. These approaches offer limited insight into how such technologies will be introduced into everyday routines and how they might transform the way people live [5].

3.2 Smart Home social impacts

Smart Home devices can manage energy use and support wellbeing, such as keeping families connected or aiding people with disabilities. However, there are many risks associated with them too. Another European Commission plan warns that rising demand for energy-hungry gadgets could outweigh efficiency improvements. Usually, these concerns are seen as technical issues that tech progress will fix. Unfortunately, there is a lack of ongoing research that deeply explores the social downsides of smart homes, treating them as complex challenges needing socially aware solutions [5].

There is a massive need for independent and long-term studies across diverse households to explore varied outcomes, ranging from actual impacts on total home energy consumption to the effects on social connections, personal security and overall wellbeing. Adding to this, understanding how the increasing reliance on connected devices affects the financial stability of families striving to keep up with the never-ending new technologies is crucial [5].

To unlock the full promise of smart homes, reducing energy use, improving day-to-day life and making stronger bonds, it is essential to view the home not just as a hub for technology or data collection, but as a vital social environment where human interactions are a priority, not technology [5].

3.3 Existing commercial products

As smart home applications have gained a lot of traction in recent years, tech giants have come out with their own interpretation of a smart home assistant. Below are listed the most globally sought-after voice assistants.

3.3.1 Alexa

Amazon's Alexa is a well-known voice-activated assistant that plays a key role in modern smart homes. It enables users to control various connected devices through simple voice commands, enhancing convenience and accessibility. Alexa's integration with a wide range of smart home products, from lighting and thermostats to security systems, allows for seamless automation and personalized experiences. On top of this, its continuous updates keep expanding its features and upgrading the user experience.

It is the leader in voice assistant sales on the market, with over 100 million devices worldwide. Priced at around 200 RON on the Romanian market, it is one of the most reliable solutions for smart home technology integration.

3.3.2 Google Nest

Google Nest represents a leading smart home ecosystem that integrates voice-activated

assistance with a variety of connected devices. Power by Google Assistant, Google Nest facilitates seamless control of home environments, including lighting, security systems, thermostats and entertainment devices. Known for their intuitive user interface and broad compatibility, Google Nest devices have gained significant popularity worldwide. Its advantage on the market is combining advanced voice recognition with integration into the wider Google ecosystem, including services such as Google Calendar or Chromecast. With roughly 95 million units sold worldwide, and costing around 400 RON in Romania, it is a strong contender to Alexa in the voice assistants market [44].

3.3.3 Apple HomePod

Apple's HomePod, being powered by the Siri voice assistant, represents a key player in the smart home market, combining high quality audio with intelligent home control. The HomePod acts as a central hub for Apple's HomeKit ecosystem, which allows users to control, in a secure manner, all their other Apple accessories, which is a big advantage as their phones, tablets or smartwatches are among the best selling in their category each year.

Its price is a bit higher compared to their market rivals, priced at around 2000 RON in Romania. However, in 2020 Apple came out with the Mini version, which has seen more popularity than its bigger and more expensive version, priced at less than half the price of the original version, at around 600 RON in Romania [45].

3.4 Conclusion

The integration of smart home systems remains uncertain due to the complexity of user engagement. Findings from expert insights, fieldwork, and academic sources highlight several key requirements for greater acceptance. Users are wary of data misuse, cyber risks and surveillance. Devices must be dependable, easy to use and built to last.

Reducing unnecessary power use is also critical, possibly through better design standards, user feedback or automated limits on excessive usage. Environmental concerns related to hardware waste and energy use by data systems must also be addressed.

Finally, steps are needed to ensure that emerging technologies are accessible to all social groups, avoiding a wider digital gap [4].

4 Theoretical fundamentals

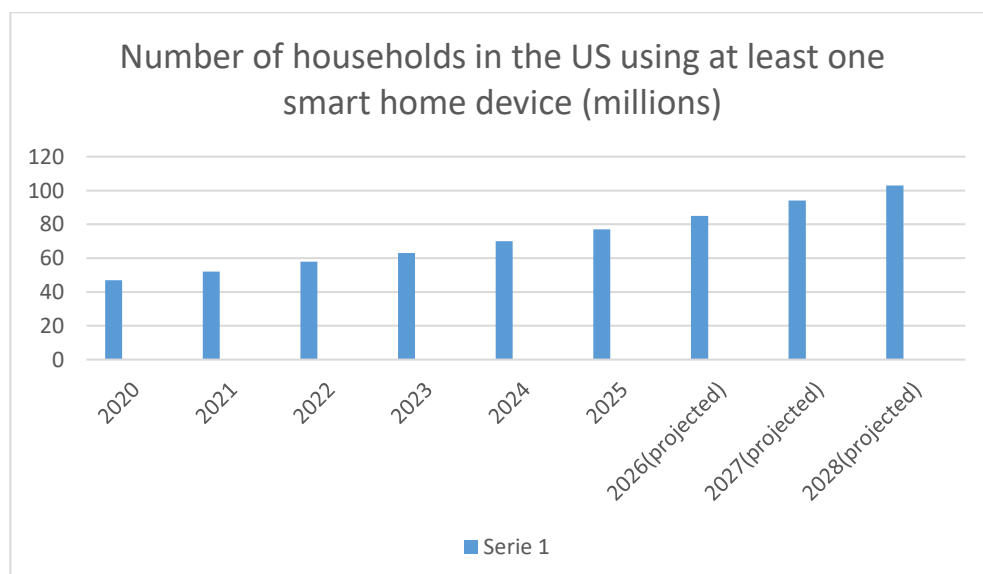
The last few years have seen smart homes evolve from a futuristic idea to something that many families worldwide cannot live without. Smart homes feature advanced technologies such as sensors, microcontrollers, wireless communication, and automation technologies to offer improved convenience, security, and energy efficiency in everyday life. Technological advancement is spurred by increasing consumer demands for effortless management of home surroundings, increased safety, and efficient use of resources.

The introduction of the Internet of Things has played a pivotal role in facilitating smart homes through the interconnection of different devices and systems, allowing them to talk to one another and operate with optimal efficiency. From remote-controlled lighting and temperature adjustments to automated security systems and networked appliances, smart homes turn ordinary houses into interactive, responsive living space (Figure 10) [37].



Figure 10 Smart Home visual representation [43]

One statistic shows that, as of 2024, in the United States, there were roughly 70 million households that used at least one smart home devices. That number is an increase of 10% compared to the statistics made in the previous year, showing the outstanding rise in popularity of smart home devices [38].



4.1 ESP32

The ESP32, presented in Figure 11, is a highly regarded, integrated microcontroller developed by Espressif Systems, designed to support a wide range of embedded and IoT applications. It usually features a dual-core Tensilica LX6 processor, integrated Wi-Fi and Bluetooth and a rich set of peripherals, such as ADCs, DACs or PWM controllers [6]. The combination of wireless communication capabilities and versatile hardware interfaces make the ESP32 suitable for projects that require real-time data processing, wireless connectivity and low power consumption. Additionally, the platform is widely supported by development environments such as Arduino IDE or PlatformIO, which makes it accessible to both enthusiasts and professionals. Its affordability, performance and flexibility have brought it much popularity among enthusiasts, but even in academic or industrial environments.

It includes a vast selection of communication protocols, such as SPI, I2C, UART, CAN or I2S, offering easy integration with sensor or any external peripherals [6].



Figure 11 ESP32 [12]

The ESP32 pinout (Figure 12) provides a very high number of general-purpose input/output, GPIO pins, which can cause the board to interact with a wide variety of sensors, actuators, and peripherals. The flexibility that sets it apart is that most pins are multifunctional, meaning they can be used for digital input/output, analog reading, PWM signal generation, I2C, SPI, or UART communication based on how they are programmed through software. However, not all pins are multifunctional, as some are input-only pins, some are the strapping pins used in boot, and certain ones are better avoided for output due to their sensitivity at startup. The board has around 30 usable GPIOs, as can be seen in Figure 3, but this may vary depending upon the specific model or module design.

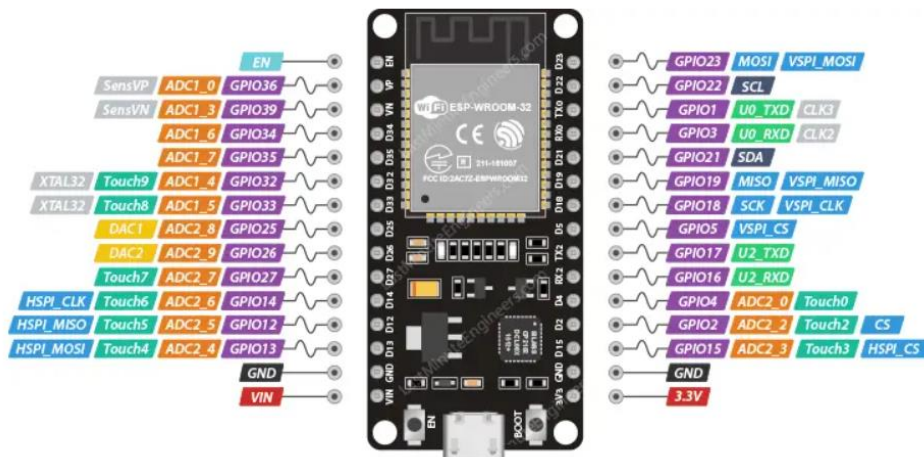


Figure 12 ESP32 Pinout [35]

4.2 ESP32-CAM

The ESP32-CAM, seen in Figure 13, is a specialized microcontroller module based on the aforementioned ESP32, designed for image capture and wireless transmission in embedded systems. It includes an OV2642 camera module, which is capable of maximum 2MP, making it suitable for low power and low cost applications, such as surveillance or image processing tasks. The board also integrates a microSD card slot for external data. Unlike other development board, it lacks a native USB interface, requiring either an external USB to serial adapter for programming it, or a middleman flashing device [7], [8].



Figure 13 ESP32-CAM [13]

4.3 Arduino Uno

Arduino Uno, seen in Figure 14, is a widely known microcontroller development board based on the ATmega328P microcontroller. It is designed for an easy use, mainly in an educational context, but also for enthusiasts. It operates at 16 MHz and includes a 32 KB of flash memory, while providing 14 digital I/O pins, along with 6 analog input channels [9]. The board features a USB type B connector for programming and serial communication, and it provides onboard voltage regulation to support both USB and external DC power at the same time. One of its strongest points is its accessibility, as its dedicated IDE, the Arduino IDE, reduces the hardware interaction, providing a high-level programming interface, a simpler version of the C/C++ language, allowing the user to freely use the microcontroller's functions, without the need to address hardware registers or any other configuration [10], [11].

While not the fastest when compared to its 32-bit and faster microcontroller clock speed competition, Arduino Uno remains a fan favorite among hobbyists and a staple in learning basic embedded systems.

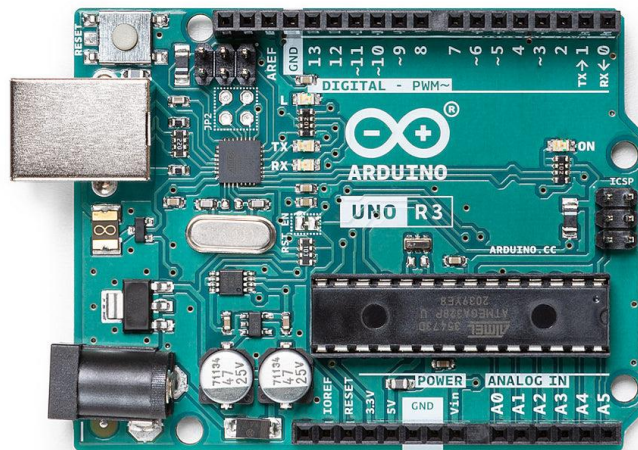


Figure 14 Arduino Uno [14]

4.4 Temperature sensor, DHT11

The DHT11, illustrated in Figure 15, is a digital temperature and humidity sensor widely used across basic IoT projects due to its simplicity, low cost and easy use. It relies on a thermistor for temperature sensing and a capacitive humidity sensor. Its compatibility with platforms like Arduino and ESP32, along with ready-to-use software libraries, makes it ideal for enthusiasts' applications. However, its limited temperature operating range of 0-50°C, as well as reduced humidity precision, in the range of 20-90%, make it less suitable for outdoor or industrial environments, and more appropriate for simple indoor projects made by enthusiasts [15].

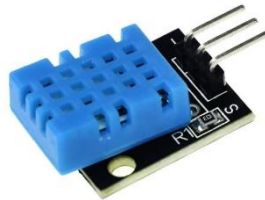


Figure 15 DHT11 sensor [16]

4.5 Fingerprint sensor module

A fingerprint sensor module, identified below in Figure 16, is a compact biometric device that captures and analyzes human fingerprint patterns to enable secure identification or access control. They include an optical or capacitive fingerprint scanner, an internal processor for image processing and onboard flash memory for enrolled fingerprints [17]. Upon receiving a scan, the image is post-processed to improve its quality and make it ready so that a certain algorithm can be applied to it. Communication with the microcontroller is usually managed over a UART interface using a standardized serial protocol [18].

While not the most cost-effective alternative on the market compared to other input devices, a fingerprint sensor offers a robust, modern and secure way of implementing a controlled biometric access to a system. Its ability to uniquely identify individuals without the need for keys or passwords make it a powerful solution for smart lock systems in IoT projects.



Figure 16 Optical fingerprint sensor module [19]

4.6 DC Motor

A DC motor, illustrated below in Figure 17, is an electromechanical device that converts direct current into rotational mechanical motion. It operates on the fundamental principle of Lorentz force, where current-carrying conductors placed within a magnetic field experience a

torque that causes the rotor to spin [20]. In embedded systems, these DC motors are used for their simplicity and ease of control, with their speed precisely regulated using PWM.

These motors are typically driven using motor circuit drivers, such as the L9110 or L298N, which act as intermediaries between the low-power control logic of microcontrollers and the higher current demands of the motor itself. Although not as precise as stepper or servo motors, DC motors remain a practical choice for applications that prioritize straightforward rotational motion, not precise small adjustments [21].



Figure 17 DC Motor [22]

4.7 Solenoid lock

A solenoid lock, visible below in Figure 18, is an electromechanical locking mechanism that relies on a solenoid, a coil of wire that acts as an electromagnet when energized, to control a mechanical latch. When current is applied, the solenoid generates a magnetic field that retracts or extends a metal plunger, thus locking or unlocking the mechanism [23].

These locks can be used in access control systems, where the triggering doesn't happen manually, but rather electronically, whether through a switch or via automation through a microcontroller.

In microcontroller based systems, solenoid locks are activated through driver circuits, transistors or relays, as their current usually exceeds the output capacity of general purpose I/O pins. Simple and effective, the solenoid lock is a brilliant alternative for integrating security in the physical form into projects or systems [24].



Figure 18 Solenoid lock [25]

4.8 Magnetic sensor

A magnetic contact sensor is a simple digital input device, used to detect proximity between two objects, usually one a static one and a moving one, based on magnetic field interaction. These kinds of sensors are made out of two parts, as seen below in Figure 19: a fixed housing that contains a switch mechanism and a magnet mounted on the mobile part. When the magnet is brought within range, the switch closes/opens, completing the circuit [26].

In embedded systems or smart projects, they are usually used for door or window monitoring, automating events based on their state. Their price, reliability and compatibility with usual microcontrollers, along with the fact that they don't need calibration, make them the staple choice in position-sensing in embedded applications.



Figure 19 Magnetic sensor [27]

4.9 Ultrasonic sensor

An ultrasonic sensor measures distance by emitting high-frequency sound waves and calculating the time it takes for the echo to return after reflecting off a surface. This process, known as time-of-flight, allows the sensor to determine the distance to nearby objects [28], [29].

They include either one or two transducers: if it's only one, it does both the job of emitting and receiving the signal, while those with two transducers, like the one seen below in Figure 20, have the jobs divided [29]. It is widely used in all kinds of electronic projects, mainly because of its accessibility, price and reliability. It's usually preferred over infrared sensor for measuring distance, as the infrared ones can provide inaccurate measurements for black objects and for smokey environments.



Figure 20 Ultrasonic sensor [30]

4.10 Relay module

A relay module, shown in Figure 21, is an electromechanical switching device that allows low voltage control systems to interface with high voltage circuits. The working principle is the following: a small current energizes an internal coil, which generates a magnetic field that mechanically triggers a switch to open/close a separated high voltage circuit [31].

These modules are widely used to control various devices inside projects. They are very compatible with standard microcontrollers such as Arduino Uno or ESP32, being mainly plug-and-play. The low price, high reliability and high accessibility make them a component that isn't usually missing from electronic or embedded projects.

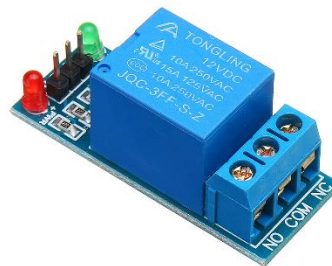


Figure 21 Relay module [32]

4.11 Motor driver

A DC motor driver, illustrated Figure 22, serves as an intermediary between a microcontroller and the motor, allowing the microcontroller to control the motor's speed and direction of rotation, without directly engaging with dangerously high current for the microcontroller. They usually use H-bridge logic in order to reverse current flow, thus making it possible for the motor to either spin forward or backward. They regulate motor speed and rotation efficiently by receiving low power signals from the microcontroller board [33].

It is a valuable component in any project that involves any kind of motor, as without it, the microcontroller would be exposed to very high currents and voltages and there's a big risk of damaging it. Easy to use, affordable and board compatible, it's a component that can't miss in any electronic project involving a DC motor.

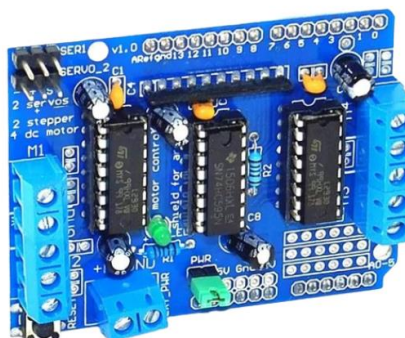


Figure 22 DC motor driver [33]

4.12 DC Fan

A DC fan, pictured in Figure 23, is an electromechanical device designed to generate airflow, often used to keep electronic components actively cool. The working principle is based on a brushless DC motor paired with some plastic blades, which produce an air current when spinning. They have a simple interface, controllable through PWM and low power consumption. They can be easily integrated into small electronic projects, compatible with microcontrollers, usually via a transistor or MOSFET [34].



Figure 23 DC fan [34]

4.13 Wireless communication in Smart Home Systems

Wireless communication bridges are a key feature on today's smart home systems. They allow for single devices within the household to share information seamlessly with one another through a wireless network, often a Wi-Fi network, without having any physical connections.

A wireless bridge (Figure 24) is essentially an intermediate node, as illustrated in Figure 15, that relays data packets among different devices or subsystems that can run on different protocols or hardware. This allows the smart home system to be modular and adaptable and to have sensors, controllers, and user interfaces on different devices [42].

The bridge facilitates communication by:

- Receiving data from one device
- Processing or converting the data if needed
- Transmitting it over a wireless network to the destination device

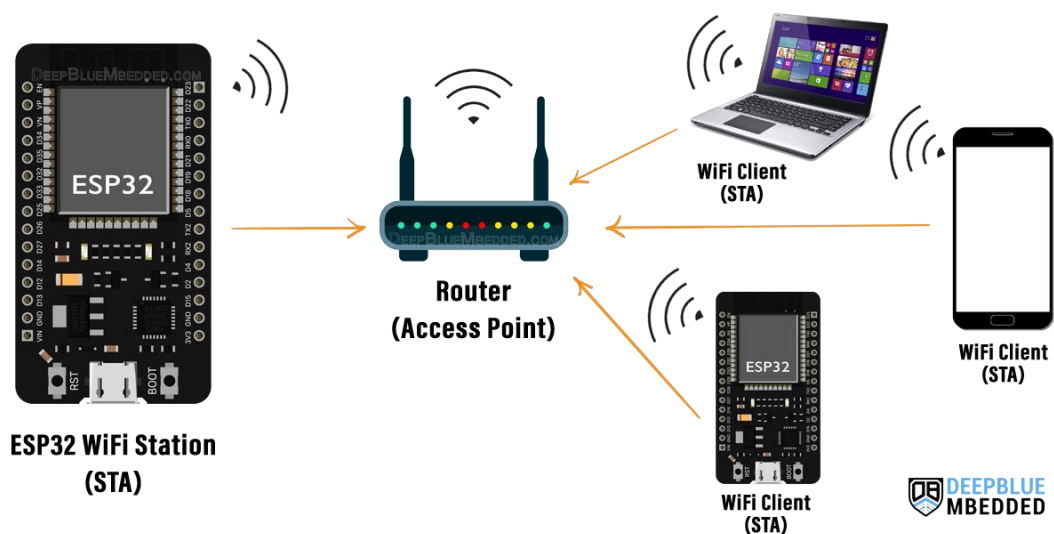


Figure 24 Wireless transmission diagram [42]

4.14 Wi-Fi as a means of communication

Wi-Fi is a widely used wireless communication technology in IoT systems because it offers exactly what the user is looking for in such a system: high data rates, reliability and compatibility with existing home networks.

In an IoT project, multiple devices connect to the same Wi-Fi network, allowing them to send and receive information. Devices often operate as clients, servers or even both, depending on their role in the system.

However, communication over Wi-Fi in indoor environments faces several drawbacks as well, such as:

- Signal attenuation by walls or furniture
- Interference by other electronic devices and networks that can disrupt communication
- Latency, as fast data delivery is essential in these systems

4.15 HTTP Protocol in IoT systems

At a higher level, devices in a smart home often communicate using standard protocols like HTTP which is fundamental to web communications. HTTP enables devices to exchange messages that command actions or update statuses. Three of the most important HTTP methods used in IoT systems are GET, PUT and POST [36].

- GET method – is a method used to retrieve data from a server, fetch information such as webpage content or sensor status [36].
- PUT method – it is used to update or replace a resource at a specific URL. It is an idempotent method, meaning multiple identical PUT requests result in the same state [36].
- POST method – it is used to submit data to a server to create or update resources. In IoT projects, POST requests are usually used to send sensor reading, authentication credentials or user commands. POST is not an idempotent method, meaning repeated identical requests can result in different effects [36].

4.16 Network Architecture and Communication Protocol

The architecture of this IoT system combines an ESP32-CAM, a 30-pin ESP32 and an Arduino Uno, which handles an optical fingerprint module, using both wireless and serial communication for secure smart home automation. The ESP32-CAM acts as the server for the web interface, while the 30-pin ESP32 handles the sensors and actuators via Wi-Fi (HTTP/1.1). The Arduino Uno sends a simple message suggesting the access status to the 30-pin ESP32, with the necessary voltage regulations for safe 5V to 3.3V communication. The client in this situation is the user using a web browser (Figure 25).

This design isolates critical tasks: the Arduino focuses solely on biometric verification, while the 30-pin ESP32 processes results, controls hardware and syncs states across the network. The serial interface provides immediate authentication updates, while sensor data is being transmitted at regular intervals. This multi-layered approach combines the reliability of wired serial communication for security functions with the flexibility of wireless networking for sensor data and control.

The ESP32-CAM operates as a standalone HTTP server providing real-time video over Wi-Fi using the Motion JPEG protocol and directly compatible with standard web browser via a multipart HTTP link. The live feed is available on the web server after and only after the fingerprint module delivers the validation message, completely restricting unauthorized access. This setup eliminates any reliance on outside networks. While the M-JPEG protocol is very stressful on the processor

and it may downgrade its performance, the live feed is provided only to the request of the user, not continuously, and it offers predictable latency and an effective and modern security model.

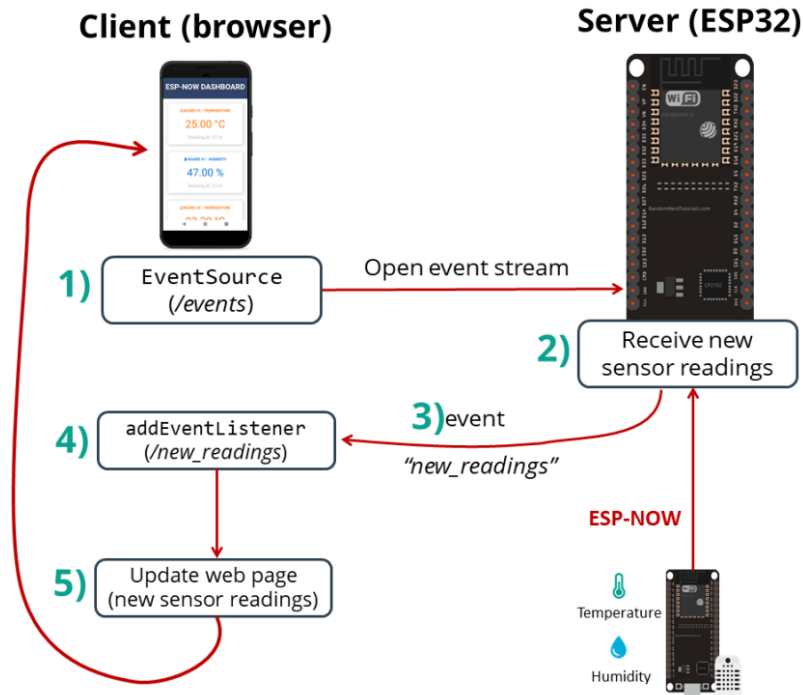


Figure 25 General architecture and communication of an IoT system [39]

4.17 Potential Threats and Vulnerabilities

One important aspect of IoT systems in general is the privacy and security of the setup. Despite both the wired and wireless connections in the setup of this project, it does have some notable flaws in the security aspects.

Although essential features remain inaccessible without fingerprint validation, the Wi-Fi interface is still susceptible to HTTP traffic interception, which may allow foreign control functions or video stream hijacking.

One more notable vulnerability aspect is the M-JPEG live feed not being encrypted, exposing it to potential attackers once turned on. However, a strong obstacle to an attack of this kind is the biometric validation barrier, which can be a real challenge to compromise.

This architecture is well suited to circumstances where physical security is a guarantee because it places a higher priority on defense against distant network attacks while assuming localized hardware risks.

Estimates show that up to 80% of smart home systems are actually prone to cyber attacks. These cyber attacks are of many types (Figure 26), depending on the architecture of the systems [40].

- Man-in-the-middle: an attacker can interfere with sensor data or hijack control systems, turning it into an operational chaos and posing safety threats [40].
- Identity theft: unsecured smart devices can leak personal data causing identity thefts [40].

- Device hijacking: attackers can easily hijack one or more devices to control entire systems [40].
- DoS/DDoS: Denial of Service (DoS) or Distributed Denial of Service overwhelm systems to disrupt access [40].

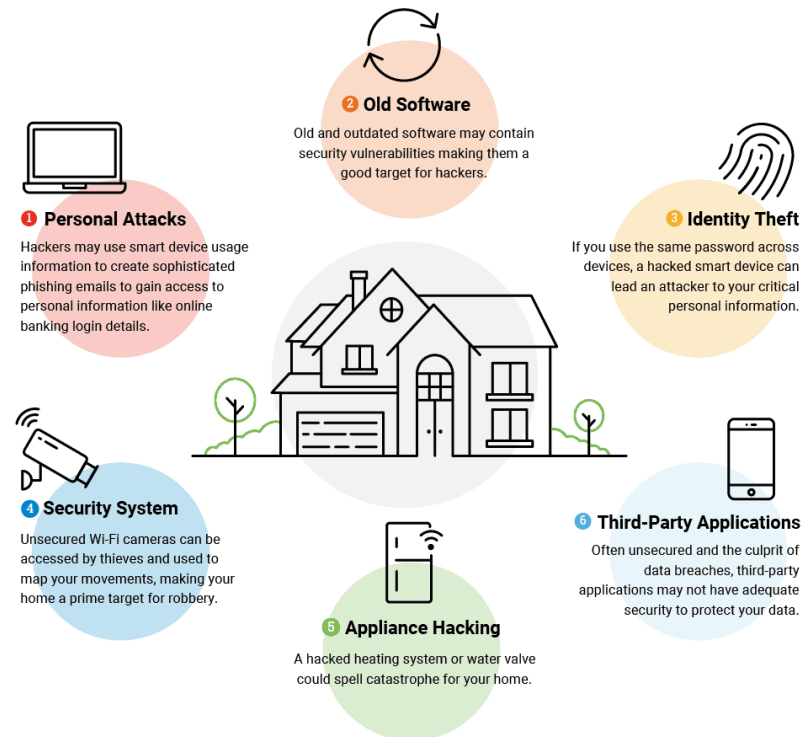


Figure 26 Smart home security risks [41]

5 Implementation

This chapter details the integration of hardware components, communication protocols and the software logic used to create a fully functional smart home system. It covers the interaction between devices, user interface and logic flows based on the block diagram.

5.1 System Architecture and Block Diagram

The smart home system architecture is divided into three main components: the ESP32-CAM, the ESP32 30-pin microcontroller, and the Arduino Uno (Figure 27). The ESP32-CAM acts as the user interface host and security camera module, while the ESP32 30-pin is responsible for managing the sensors and actuators. Communication between the two boards happens exclusively wirelessly, over a Wi-Fi bridge.

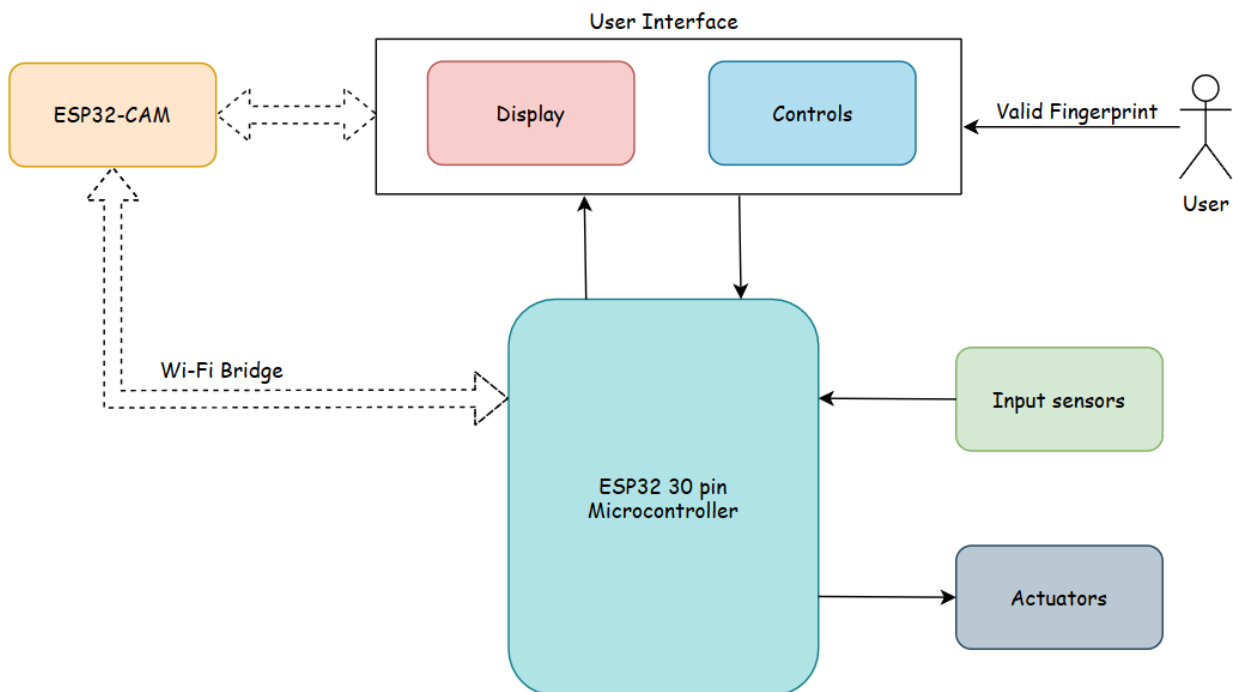


Figure 27 Block Diagram of the proposed solution

- **ESP32-CAM**

The ESP32-CAM hosts the web server that delivers the user interface, providing a live video stream from the onboard camera alongside real-time data from the connected sensors. Additionally, it enables wireless communication by sending commands to the ESP32 30-pin board via WiFi.

- **ESP32 30-pin**

The 30-pin ESP32 manages all connected sensors and actuators, executing commands received from the ESP32-CAM. It also transmits sensor status updates back to the ESP32-CAM to maintain a synchronized system operation.

- **User Interface**

The user interface is accessible through a web browser, with the server hosted directly by the ESP32-CAM. User access is restricted until authentication is completed via a valid

fingerprint. Once authenticated, the interface displays all the system's states, including the status of doors and windows, the garage door status, the ambient temperature and lighting conditions. Furthermore, the user can control the garage door and also lock and unlock the door.

- **User**

The user interacts with the system by scanning a fingerprint. Upon successful validation, access is granted to the user interface, enabling further interaction with the system's features.

5.2 Flashing the ESP32-CAM

For starters, for this project the ESP32-CAM board had to be purchased. However, it came as a surprise that it lacked USB ports to power it on or to upload code through. While additional research could've prevented this oversight, the most common way to power and flash it is through an external FTDI programmer (Figure 28), pictured below, which had not been identified as a requirement prior to the purchase of the board.

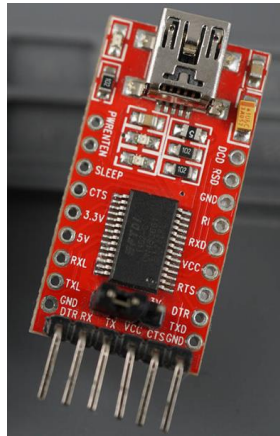


Figure 28 FTDI Programmer [46]

Further research was conducted to find an alternative method, and one such method was identified. This approach, similar to the FTDI programmer method, uses an Arduino board as the flashing device. Given that the Arduino Uno board was already available, this method was adopted, avoiding the need to increase the cost of the project with the purchase of the FTDI programmer.

The required hardware connections for flashing the ESP32-CAM were simulated using the Fritzing software. As illustrated in Figure 29 below, the connections are as follows:

- Arduino Uno 5V → ESP32-CAM 5V
- Arduino Uno GND → ESP32-CAM GND
- Arduino Uno RST → Arduino Uno GND
- Arduino Uno Tx → ESP32-CAM U0T
- Arduino Uno Rx → ESP32-CAM U0R
- ESP32-CAM GPIO0 → ESP32-CAM GND

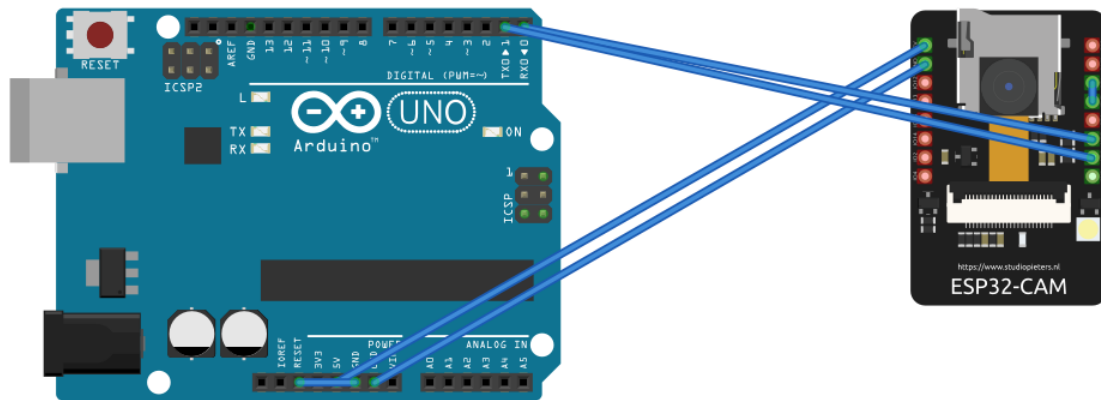


Figure 29 Wiring for flashing the ESP32-CAM

This way, no additional programmer is needed, and all can be done within the Arduino IDE, specifically version 1.8.12. The following settings need to be done in order for this to function properly.

- From the navigation bar, select Tools → Board → and install the “esp32” library by Espressif Systems (Figure 30)

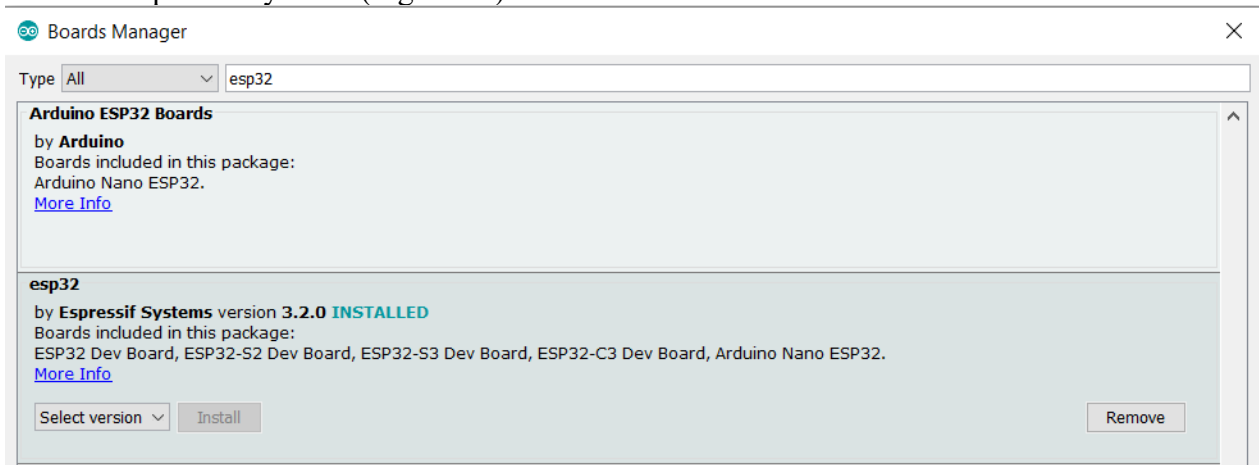


Figure 30 Board to install in Arduino IDE for flashing ESP32-CAM

- Set the upload speed to 115200
- Set the flash frequency to 40 MHz
- Select the correct port where Arduino is plugged in (can be checked in Device Manager)
- In the Programmer section, select “AVR ISP”, this is the main reason I specified an older version of Arduino IDE, as the newer versions have this option removed, and therefore this operation cannot be performed, or I have not been successful at least (Figure 31)

Auto Format	Ctrl+T
Archive Sketch	
Fix Encoding & Reload	
Manage Libraries...	Ctrl+Shift+I
Serial Monitor	Ctrl+Shift+M
Serial Plotter	Ctrl+Shift+L
WiFi101 / Wi-FiNINA Firmware Updater	
Board: "ESP32 Wrover Module"	>
Upload Speed: "115200"	>
Flash Frequency: "40MHz"	>
Flash Mode: "QIO"	>
Partition Scheme: "Huge APP (3MB No OTA/1MB SPIFFS)"	>
Core Debug Level: "None"	>
Erase All Flash Before Sketch Upload: "Disabled"	>
Port	>
Get Board Info	
Programmer: "AVR ISP"	>
Burn Bootloader	

Figure 31 Settings to be made for the flashing

Now, the only thing left is to write the code and hit the Upload button. Because of the flashing device, the Arduino Uno, and the fact that an older version of the Arduino IDE is used, the compiling and uploading of the code will happen at a slower rate, compared to uploading code directly to Arduino Uno for example, with the latest Arduino IDE version, which happens very fast.

Once the upload is done, the ESP32-CAM needs to be reset. The RST button is on the upper back of the board, but the reset will happen if and only if the ESP32-CAM GPIO0 is disconnected from GND.

5.3 Integrating the fingerprint module

The fingerprint sensor serves as the primary authentication method in the smart home system. Access to all the system's features is permitted if and only if a valid fingerprint can be provided. It communicates the authentication's result to the Arduino Uno which communicates it further to the ESP32 30-pin.

Wiring connections can be seen in Table 2 below.

Fingerprint module (colors may vary with the model)	Arduino Uno
VCC (green)	3.3 V
GND (red)	GND
TXD (yellow)	Digital Pin 4
RXD (black)	Digital Pin 5

Table 2 Fingerprint module connections

5.3.1 Code explanation

The fingerprint sensor is controlled via the Adafruit Fingerprint Sensor Library, which manages communication and search operations, over a software serial interface, which allows the usage of digital pins 4 and 5 as serial TX/RX (Figure 32).

```
#include <SoftwareSerial.h>
#include <Adafruit_Fingerprint.h>

SoftwareSerial mySerial(4, 5); // RX, TX for fingerprint sensor
Adafruit_Fingerprint finger(&mySerial);
```

Figure 32 The necessary libraries to communicate and handle the fingerprint sensor

The communications are initialized in the *Setup()* function (Figure 33) for both the PC and the fingerprint sensor. It then starts the fingerprint sensor and checks if it responds using the *finger.verifyPassword()* function. If the connections are done correctly, it will print “Fingerprint sensor found!”, otherwise it will print “Fingerprint sensor NOT found!” to let the user know that something is faulty in the communication.

```
void setup() {
    Serial.begin(9600);
    mySerial.begin(57600);
    finger.begin(57600);

    delay(1000);

    if (finger.verifyPassword()) {
        Serial.println("Fingerprint sensor found!");
    } else {
        Serial.println("Fingerprint sensor NOT found!");
        while (1) delay(100); // halt if sensor not found
    }
}
```

Figure 33 The *Setup()* function initializing the communications and checking the response of the fingerprint sensor

The *loop()* function (Figure 34) continuously checks for a fingerprint. It starts with *finger.getImage()* to detect if a finger is placed on the sensor or not. If no finger is detected, it waits briefly and exits.

If a finger is detected, it tries to convert it into the function *image2Tz()*. If the conversion fails, access is denied. If the conversion is a success, it goes on to *finger.fingerSearch()* function, trying to find a match with the already existing fingerprints, the ones enrolled in the sensor’s flash memory. If there is a match, it prints the ID and allows access by calling *sendAccessStatus(true)*. If there is no match found, it calls *sendAccessStatus(false)*.

```

void loop() {
    uint8_t p = finger.getImage();

    if (p == FINGERPRINT_NOFINGER) { // No finger detected, do nothing, no spam
        delay(100);
        return;
    }

    if (p != FINGERPRINT_OK) {
        Serial.println("Error reading finger image");
        sendAccessStatus(false);
        delay(1000);
        return;
    }

    p = finger.image2Tz();
    if (p != FINGERPRINT_OK) {
        Serial.println("Failed to convert image");
        sendAccessStatus(false);
        delay(1000);
        return;
    }

    p = finger.fingerSearch();
    if (p == FINGERPRINT_OK) {
        Serial.print("Fingerprint ID found: ");
        Serial.println(finger.fingerID);
        sendAccessStatus(true);
    } else if (p == FINGERPRINT_NOTFOUND) {
        Serial.println("Fingerprint not recognized");
        sendAccessStatus(false);
    } else {
        Serial.print("Fingerprint search error: ");
        Serial.println(p);
        sendAccessStatus(false);
    }
}

```

Figure 34 The loop() function implementing continuous fingerprint detection and authentication

5.4 Integration of the main controller, ESP32 30-pin

This chapter focuses on the “brain” of the entire smart home system, the ESP32 30-pin. While the ESP32-CAM handles the web server and the camera and the Arduino Uno handles the fingerprint module, the 30-pin ESP32 acts as the main coordinator of the whole system, handling multiple tasks, such as operating the peripherals, doing both the acquisition and actuation tasks.

5.4.1 Code explanation

In the beginning of the code, the inclusion of several libraries (Figure 35) is absolutely key in order to be able to perform certain tasks throughout the code.

- WiFi library – this is needed to connect the device to a wireless network
- HTTPClient library – this is needed for communication to be possible between the ESP32 30-Pin and ESP32-CAM through HTTP requests
- DHT library – this enables reading the temperature from the sensor
- WebServer library – this allows for the creation of a local web server to handle the aforementioned HTTP requests
- Preferences library – this is used to save configuration data like settings in order to be persistent even in the event of a power loss

The network’s credentials are vital and need to be updated according to which network it needs to be connected to. Also necessary is the ESP32-CAM’s IP address, in order to establish the connection with the main controller. This is dependent on the network, so it does change once the network is also changed.

The selected GPIO pins do not have a special reasoning, except for their functionalities, for

example, for the LEDs that need PWM adjustments, the pin selected needs to be able to support PWM.

```
#include <WiFi.h>
#include <HTTPClient.h>
#include <DHT.h>
#include <WebServer.h>
#include <Preferences.h>

WebServer localServer(80);
Preferences preferences;

// Pin Definitions
#define GARAGE_MOTOR_PIN_1 14
#define GARAGE_MOTOR_PIN_2 27
#define ULTRASONIC_TRIG 13
#define ULTRASONIC_ECHO 12
#define DOOR_REED_PIN 32
#define WINDOW_REED_PIN 33
#define DHTPIN 25
#define DHTTYPE DHT11
#define COOLING_LED 26
#define HEATING_LED 15
#define SOLENOID_PIN 4
#define PWM_LED_PIN 21

// WiFi and Server
const char* ssid = " ";
const char* password = " ";
const char* esp32CamIP = "192.168.1.183";
```

Figure 35 Necessary libraries and used GPIOs

The *setup()* function (Figure 36) starts by initializing two serial ports, one for debugging messages and one for communicating with the Arduino Uno, on RXD2 and TXD2. Next, it opens up a *Preferences* storage section called "smart home", which acts like a small built-in file system. It loads up saved values for the likes of light brightness, the two temperature thresholds and other flags necessary later in the code. These “settings” remain ever after powering it off.

After this, the GPIO pin’s roles are set up. Pins linked to the garage motor through the driver, and the solenoid lock are all set as outputs. The door and window magnetic sensors, which detect if they are open or closed, are configured as inputs with internal pull-up resistors. The ultrasonic sensor is set up here as well, with its TRIG pin being treated as an output in order to send pulses, and the ECHO pin being an input, to receive the bouncing waves coming back.

```
void setup() {
  Serial.begin(115200);
  Serial2.begin(9600, SERIAL_8N1, 16, 17);

  // Initialize preferences
  preferences.begin("smart-home", false);
  pwmValue = preferences.getInt("pwm", 128);
  coolingThreshold = preferences.getInt("coolThresh", 27);
  heatingThreshold = preferences.getInt("heatThresh", 22);
  accessGranted = false;
  preferences.putBool("access", false); // Reset on every startup

  // Initialize pins
  pinMode(GARAGE_MOTOR_PIN_1, OUTPUT);
  pinMode(GARAGE_MOTOR_PIN_2, OUTPUT);
  pinMode(COOLING_LED, OUTPUT);
  pinMode(HEATING_LED, OUTPUT);
  pinMode(SOLENOID_PIN, OUTPUT);
  digitalWrite(SOLENOID_PIN, accessGranted ? HIGH : LOW);

  pinMode(DOOR_REED_PIN, INPUT_PULLUP);
  pinMode(WINDOW_REED_PIN, INPUT_PULLUP);
  pinMode(ULTRASONIC_TRIG, OUTPUT);
  pinMode(ULTRASONIC_ECHO, INPUT);

  ledcAttach(PWM_LED_PIN, 5000, 8);
  ledcWrite(PWM_LED_PIN, pwmValue);

  dht.begin();
}
```

Figure 36 Initialization routine within the *Setup()* function, configuring serial communication, persistent storage, and I/O pins

The Wi-Fi network connects using the hardcoded SSID and password, printing dots until the connection is established completely. Once connected, it calls *notifyCamAccess(false)*, meaning that access is denied by default when the system boots (Figure 37).

```
// Connect to WiFi
WiFi.begin(ssid, password);
while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
}
Serial.println("\nWiFi connected");
notifyCamAccess(false);
```

Figure 37 WiFi initialization and connection logic

In the end, it sets up a web server endpoint called “/cmd” which listens for updates under the form of commands regarding any changes with the peripherals. Each command updates the correct variable and stores it in flash, so it is remembered. When the command handler is ready, the local server starts with this call *localServer.begin()*, and this way, the microcontroller can accept HTTP requests (Figure 38).

```
localServer.on("/cmd", HTTP_GET, []() {
    if (localServer.hasArg("cool") && localServer.hasArg("heat")) {
        coolingThreshold = localServer.arg("cool").toInt();
        heatingThreshold = localServer.arg("heat").toInt();
        preferences.putInt("coolThresh", coolingThreshold);
        preferences.putInt("heatThresh", heatingThreshold);
        Serial.println("Received thresholds - Cooling: " + String(coolingThreshold) + ", Heating: " + String(heatingThreshold));
    }

    if (localServer.hasArg("light")) {
        pwmValue = localServer.arg("light").toInt();
        preferences.putInt("pwm", pwmValue);
        setPWM(pwmValue);
        Serial.println("Received light PWM value: " + String(pwmValue));
    }

    if (localServer.hasArg("lock")) {
        Serial.println("Received LOCK command for door solenoid");
        setSolenoid(false); // Lock the door
    }

    if (localServer.hasArg("unlock")) {
        Serial.println("Received UNLOCK command for door solenoid");
        setSolenoid(true); // Unlock the door
    }

    if (localServer.hasArg("openGarage")) {
        Serial.println("Received OPEN GARAGE command");
        openGarage();
    }

    if (localServer.hasArg("closeGarage")) {
        Serial.println("Received CLOSE GARAGE command");
        closeGarage();
    }

    localServer.send(200, "text/plain", "OK");
});
localServer.begin();
```

Figure 38 Setting up the /cmd endpoint

Each cycle, the *loop()* function (Figure 39) checks for serial input from the fingerprint system in case of a valid fingerprint being input. It then handles any incoming web requests via the web server.

After that, it updates sensor readings by calling the *sensorUpdates()* function. In the end, there is a special, specific check for the garage auto-close function; if it is triggered, it checks if the countdown timer has expired through the *millis()* function. If it did expire, it closes the garage and resets the flag, so it doesn't run again.

```

void loop() {
    checkSerialFingerprint();
    localServer.handleClient();
    updateSensors();
    if (autoCloseTriggered && (millis() - autoCloseStartTime >= AUTO_CLOSE_DELAY)) {
        Serial.println("Auto-close timer expired. Closing garage.");
        closeGarage();
        autoCloseTriggered = false;
    }
}

```

Figure 39 The `loop()` function to handle fingerprint check continuously

There are auxiliary functions in the code that handle specific parts of the smart home system. These include functions like managing the temperature, checking the response of the ultrasonic sensor, triggering the garage door and others. Their implementations are not explained in detail here, as they follow standard embedded programming patterns and are mainly procedural.

The final relevant function in the ESP32 30-pin's code is the `sendSensorDataToCam()` function (Figure 40). As the name suggests, it packages and sends real-time sensor data to the ESP32-CAM. It builds a full HTTP GET request URL by appending query parameters to it, states and different data from different sensors. Each value is formatted into a readable string, even handling isolated cases like NaN (Not a Number).

It then initializes a `HTTPClient`, opens a connection with the ESP32-CAM using that URL and sends the requests. The timeout is intentionally kept low to avoid delays, and caching is not enabled to make sure it receives fresh data, after which it closes the connection.

```

void sendSensorDataToCam() {
    HTTPClient http;
    String url = "http://" + String(esp32CamIP) + "/update?";
    url += "door=" + String(doorOpen ? "open" : "closed");
    url += "&window=" + String(windowOpen ? "open" : "closed");
    url += "&garage=" + String(garageOpen ? "open" : "closed");
    url += "&temp=" + (isnan(currentTemp) ? "nan" : String(currentTemp, 1));
    url += "&car=" + String(carPresent ? "1" : "0");
    url += "&pwm=" + String(pwmValue); // Add current PWM value

    http.begin(url);
    http.setTimeout(200); // Reduce timeout
    http.addHeader("Cache-Control", "no-cache");
    http.GET();
    http.end();
}

```

Figure 40 Function that sends data to ESP32-CAM

5.5 Integration of the web-hosting controller, ESP32-CAM

The ESP32-CAM board acts as the central web server and user interface hub for the smart home system, hosting a responsive web app that manages access control via fingerprint authentication, displays real-time data of sensors and provides a live feed from the camera. It communicates with the ESP32 30-pin via Wi-Fi bridge, receiving sensor information and sending user requests.

5.5.1 Code explanation

The `setup()` function (Figure 41) sets up the ESP32-CAM by initializing serial communication and retrieving saved configuration settings such as light intensity or temperature thresholds from non-volatile memory using the Preferences library. It sets up the camera parameters with the corresponding GPIO pins and parameters and initializes the camera. After that, it establishes the

Wi-Fi connection to the network, very important, **the same** Wi-Fi network that the ESP32 30-pin was connected to, so that they can communicate seamlessly. Upon connection, it establishes the HTTP routes for the web server to corresponding handler functions or lambda expressions, which manage operations such as light control, garage door manipulation and other functions.

```
// Initialize preferences
preferences.begin("smart-home", false);
pwmValue = preferences.getInt("pwm", 128);
tempLow = preferences.getInt("tempLow", 18);
tempHigh = preferences.getInt("tempHigh", 26);
accessGranted = false;
preferences.putBool("access", false);

// Camera config
camera_config_t config;
config.ledc_channel = LEDC_CHANNEL_0;
config.ledc_timer = LEDC_TIMER_0;
config.pin_d0 = Y2_GPIO_NUM;
config.pin_d1 = Y3_GPIO_NUM;
config.pin_d2 = Y4_GPIO_NUM;
config.pin_d3 = Y5_GPIO_NUM;
config.pin_d4 = Y6_GPIO_NUM;
config.pin_d5 = Y7_GPIO_NUM;
config.pin_d6 = Y8_GPIO_NUM;
config.pin_d7 = Y9_GPIO_NUM;
config.pin_xclk = XCLK_GPIO_NUM;
config.pin_pclk = PCLK_GPIO_NUM;
config.pin_vsync = VSYNC_GPIO_NUM;
config.pin_href = HREF_GPIO_NUM;
config.pin_sccb_sda = SIOD_GPIO_NUM;
config.pin_sccb_scl = SIOC_GPIO_NUM;
config.pin_pwdn = PWDN_GPIO_NUM;
config.pin_reset = RESET_GPIO_NUM;
config.xclk_freq_hz = 20000000;
config.pixel_format = PIXFORMAT_JPEG;
config.frame_size = FRAMESIZE_VGA;
config.fb_location = CAMERA_FB_IN_PSRAM;
config.jpeg_quality = 12;
config.fb_count = 1;

// Server routes
server.on("/", HTTP_GET, handleRoot);
server.on("/auth", HTTP_GET, handleAuth);
server.on("/update", HTTP_GET, handleUpdate);
server.on("/lock", HTTP_GET, []() {
    Serial.println("Sending LOCK command to main ESP32");
    sendCommandToESP32("/cmd?lock=1");
    server.send(200, "text/plain", "OK");
});

server.on("/unlock", HTTP_GET, []() {
    Serial.println("Sending UNLOCK command to main ESP32");
    sendCommandToESP32("/cmd?unlock=1");
    server.send(200, "text/plain", "OK");
});

server.on("/openGarage", HTTP_GET, []() {
    Serial.println("Sending OPEN GARAGE command to main ESP32");
    sendCommandToESP32("/cmd?openGarage=1");
    server.send(200, "text/plain", "OK");
});

server.on("/closeGarage", HTTP_GET, []() {
    Serial.println("Sending CLOSE GARAGE command to main ESP32");
    sendCommandToESP32("/cmd?closeGarage=1");
    server.send(200, "text/plain", "OK");
});

server.on("/light", HTTP_GET, handleLight);
server.on("/thresholds", HTTP_GET, handleThresholds);
server.on("/camera/on", HTTP_GET, handleCameraOn);
server.on("/camera/off", HTTP_GET, handleCameraOff);
server.on("/status", HTTP_GET, handleStatus);
server.on("/stream", HTTP_GET, handleStream);
```

Figure 41 The setup() function of the ESP32-CAM

The *handleRoot()* function defines the main web interface served and hosted by the ESP32-CAM, using a raw string literal to embed the HTML, CSS and JavaScript directly within the source code.

The HTML structure begins with standard metadata, including a responsive and mobile-friendly viewport configuration. External resources are imported such as fonts, like Roboto from Google Fonts, and icon instruments, like Font Awesome.

CSS variables declared in the “:root” selector define a simple yet modern color palette, with primary, accent, success and danger tones. Global styles are applied to normalize the layout and implement a gradient background.

The content is wrapped in a “*.container*” class, centered and constrained in width to fit any screen sizes.

From a functional point of view, the interface is divided into several sections. The authentication gate initially displays a fingerprint authentication prompt, inviting the user to provide a valid fingerprint to the sensor. Upon successful validation, it transitions to the main control interface, which consists of multiple card components that are split logically based on their role: Home status, Lighting control, Climate control and Security Control.

The Climate Control feature allows the user to set temperature limits for heating/cooling systems with clear graphical indication of the selected settings. The Security Controls feature provides button-driven interfaces to lock/unlock door and garage control, along with confirmation dialogs for potentially dangerous operations like accidental closure of the garage, and the possibility to check the security camera.

The `<script>` part of the code is written in JavaScript and is crucial for implementing crucial functions of the system, such as:

- Continuous status updates via periodic AJAX requests to a `'/status'` endpoint
- Real-time control with card-like interfaces
- Proper state management for control elements, like disabling buttons when a certain action is not possible
- Network optimization for control inputs to prevent excessive request which might slow the web app down

The web application implementation resembles the ones of real-life smart products, with a clean, modern, smooth and card-based UI, featuring intuitive controls and real-time status updates. The design has a very well-chosen color palette, with smooth animations and a responsive layout. The fingerprint authentication provides top-notch security, while the mobile adaptability adopts the industry standard and along with the intuitiveness and modern features, it makes the UI/UX polished, professional, and on par with existing commercial products.

In the code there is also a set of HTTP request handler functions which enable communication between the web interface and the smart home infrastructure. Each function processes specific requests from the client, extracts relevant parameters from the URL and update system variables in real time. Their implementation is standard in embedded systems and mainly procedural, so the focus will not be on their implementation, as they are very similar to one another and standard in implementation. Some functions trigger commands that are forwarded to the ESP32 30-pin over Wi-Fi bridge, to control physical components, while others control internal states or configuration values. These handlers form the main interface through which the web server interprets user input and dictates coordinated actions throughout the entire system.

Here is how the interface looks on different screens:

- **Desktop screen size (Figure 42, Figure 43)**

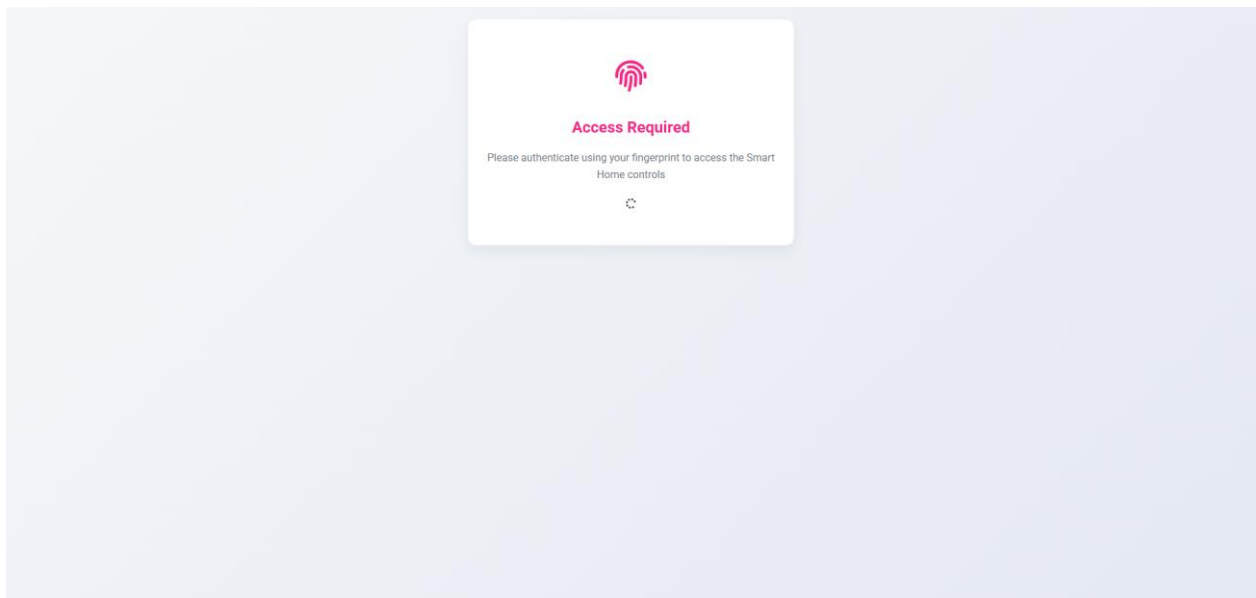


Figure 42 Desktop view - Locked Web App

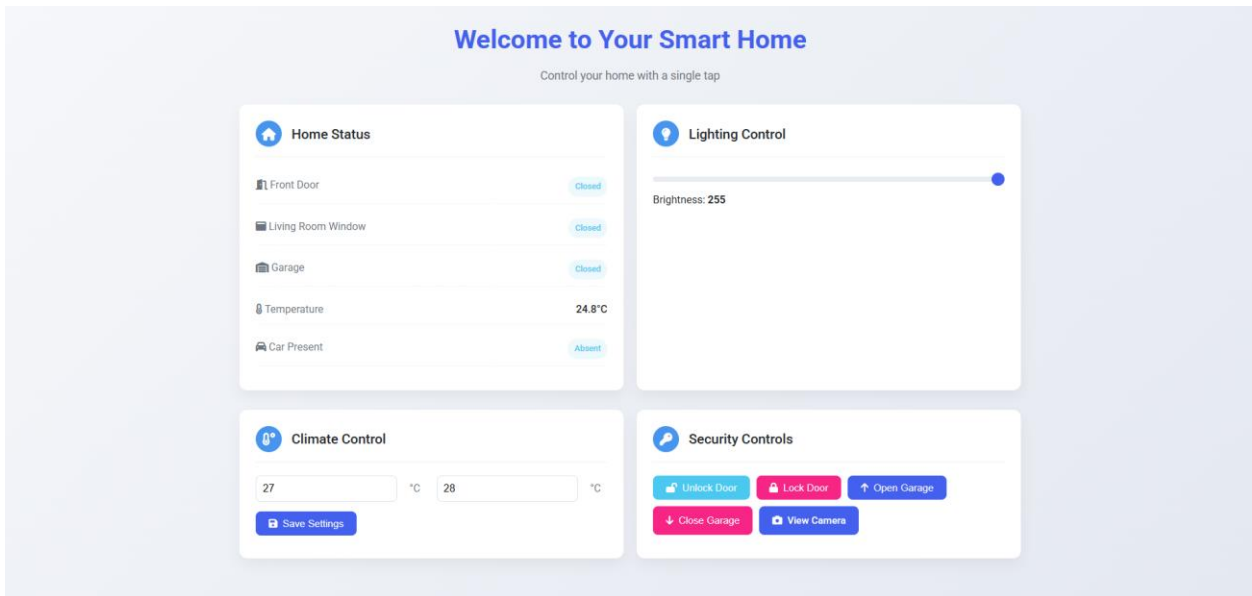


Figure 43 Desktop view - Unlocked Web App

- **Mobile screen size (Figure 44, Figure 45)**

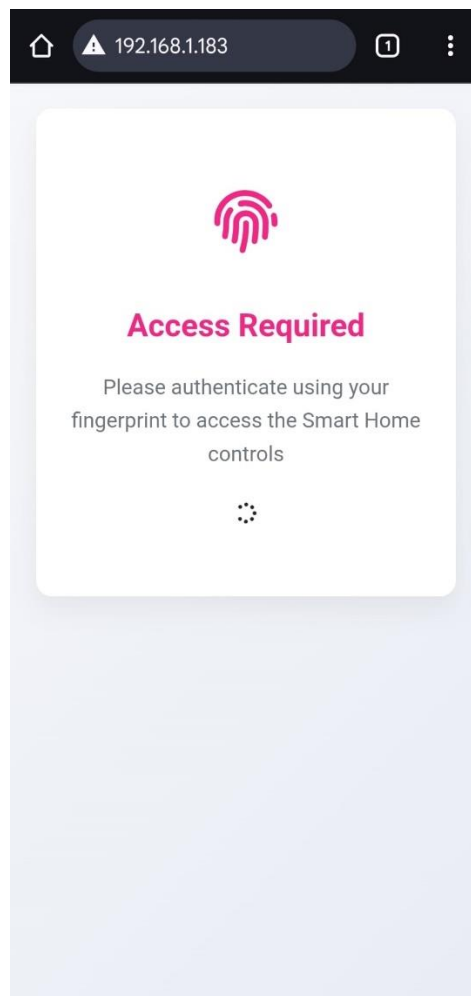


Figure 44 Mobile view - Locked Web App

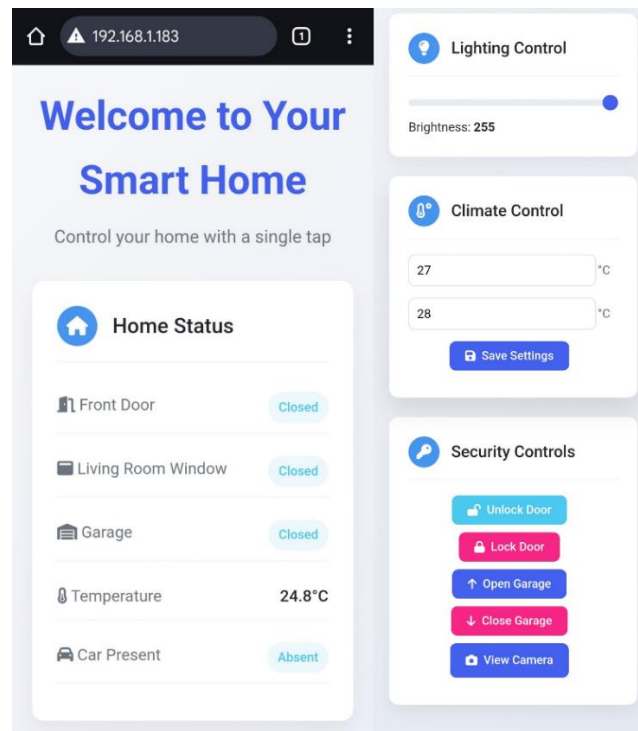


Figure 45 Mobile view - Unlocked Web App

5.6 Hardware Implementation

5.6.1 Methodology and Limitations

The physical implementation of this smart home system, the wiring and circuitry, are of an elevated level of complexity, such that a schematic representation in dedicated CAD softwares such as Fritzing, would prove to be impractical, the overall schematic would be unintelligible. Given the high component density, including microcontrollers, sensors, actuators, relays, drivers, a fully detailed diagram would become “visually chaotic” and unreadable. Adding to this, certain components are not available in any library of the software, leaving with the choice of either inaccurate substitutions or manual symbol creation, both of which may hinder with the schematic’s fidelity.

To ensure both accuracy and clarity, the practical implementation is documented through a structured textual description, accompanied by a comprehensive wiring table. This way, a clearer view of pin mapping and component wiring is provided, leaving no space for interpretation.

5.6.2 Hardware Implementation – Overview

The smart home system uses a distributed hardware architecture that optimizes computational load distribution and enhances security. The Arduino Uno acts as a security gate, managing the fingerprint sensor and another important aspect, acts as a 3.3V – through the 3.3V pin, 5V – through the 5V pin and 12V – through the VIN pin when powered with the barrel connector, power supply, as the different sensors operate at different voltages.

The ESP32 30-pin acts as the main sensor and actuator hub, overseeing almost all the functionalities: garage door operation with the DC motor, via the L9110 motor driver, the ultrasonic sensor used for the automatic closing of the garage door, collects data from the DHT11 and processes it and based on the temperature, turns on either heating, cooling or neither, state of the

magnetic sensors and also the PWM-controlled interior lighting, all of these being handled by appropriate GPIO pins, as not all pins are PWM-capable for example.

The ESP32-CAM is responsible for hosting the web-server, handling and sending HTTP requests and providing the live feed of the camera. From a connection point of view, it only needs to be powered on and nothing else, after the final code has been updated, as it handles no sensors or actuators and the communication only happens wirelessly.

One small yet important detail in the implementation is the fact that the two physically connected boards, the Arduino Uno and ESP32 30-pin operate at different logic levels, Uno at 5V logic levels and ESP32 at 3.3V. To effectively tackle this issue, a voltage divider is used on the Uno's TX line to safely step down the signal for the ESP32's RX pin. To implement the voltage divider, the simplest and most common way was chosen, two resistors of values 1 k Ω and 2 k Ω . Equations (1) and (2) outline the calculations carried out for this purpose.

$$R_1 = 1 \text{ k}\Omega ; R_2 = 2 \text{ k}\Omega \quad (1)$$

$$V_{out} = V_{in} \frac{R_2}{R_1 + R_2} = 5V \times \frac{2 \text{ k}\Omega}{1 \text{ k}\Omega + 2 \text{ k}\Omega} = 3.33V \quad (2)$$

This way, the information about the status of the fingerprint validation sent via serial UART by the Arduino Uno is safely transmitted and safely received by the ESP32. If it weren't for the voltage divider, the high voltage could have damaged the ESP32.

Given the fact that the two boards communicate and exchange signals, a critical detail for the implementation is the fact that they need to share a common ground, ensuring the voltage signals are correctly referenced.

5.6.3 Physical Implementation – Wiring Schemes

The wiring tables will be done in two phases, in order to make it clearer. Phase 1, which can be seen in Table 3 will be the connections for the fingerprint handling and communication of the result, between the Arduino Uno and ESP32 30-pin.

Device	Pin Name	Connection
Arduino Uno	RX (Pin 0)	TX2 (GPIO17) – ESP32
Arduino Uno	TX (Pin 1)	RX2 (GPIO16) – ESP32 (via voltage divider)
Arduino Uno	Digital Pin 4	TXD – Fingerprint Sensor
Arduino Uno	Digital Pin 5	RXD – Fingerprint Sensor
ESP32	RX2 (GPIO16)	TX – Arduino Uno
ESP32	TX2 (GPIO17)	RX – Arduino Uno
Fingerprint Sensor	VCC	3.3V – Arduino Uno
Fingerprint Sensor	GND	GND – Arduino Uno
ESP32	GND	GND – Arduino Uno

Table 3 Phase 1 connection table

The table below (Table 4) represents Phase 2 of the implementation wiring scheme, containing all the peripherals, sensors, actuators, drivers, motors, LEDs etc. handled by the main controller of the whole system, the 30-pin ESP32.

It is worth noting that the solenoid lock and the DC fan are both driven by relays. As the voltage required by the solenoid lock is way higher than the 5V, the only solution to obtain that high voltage without an external power supply was to wire it to the VIN pin of the Arduino Uno, as if it's powered with the barrel connector, it can provide up to 12V which is sufficient for the solenoid lock.

After this, the DC fan was connected via a 5V relay powered by the Arduino Uno as well, and the fan was connected such that it received 5V from the Uno via the relay's switching terminals, COM and NO. This setup led to a massive instability problem, as the current drawn by the solenoid and the fan mainly was excessive. The voltage regulator of the board was unable to maintain sufficient voltage under load, leading to it rebooting constantly.

The issue was solved not needing an external power supply, but by making use of the ESP32's VIN Pin, which was connected to the NO terminal of the relay. The VIN Pin provides a stable 5V source from the USB-C connection. This improved variation reduces the stress of the Uno, while not affecting the integrity of the ESP32.

Component	ESP32 Pin	GND	VCC
Motor Driver A-1A	GPIO14	GND	5V Rail
Motor Driver A-1B	GPIO27	-	-
Ultrasonic Sensor TRIG	GPIO13	GND	5V Rail
Ultrasonic Sensor ECHO	GPIO12	-	-
Entrance Door Magnetic Sensor	GPIO32	GND	-
Window Magnetic Sensor	GPIO33	GND	-
DHT11 Sensor Data Pin	GPIO25	GND	5V Rail
Interior Lighting LED	GPIO21	GND	
Relay + Solenoid Door Lock	GPIO4	GND	Arduino Uno VIN Pin (NO relay terminal)
Heating LED	GPIO15	GND	-
Relay + DC Fan	GPIO26	GND	ESP32 VIN Pin (NO relay terminal)

Table 4 Phase 2 connection table

6 Experimental Results

In this chapter, several tests have been conducted on the smart home's functionalities in order to observe the system's capabilities and behavior in reaction to different inputs and to evaluate its promptitude and efficiency. The tests will consist of situation like turning the interior lighting on at different intensities using the slider, opening the window and the door and check the statuses, set different temperature thresholds in order to turn the cooling on, then the heating on and leave it in the dead-zone, open the garage door and simulate the leaving and arriving of the car and others.

6.1 Window Monitoring

The window status is determined using a magnetic sensor, handled by the 30-pin ESP32. When the window is closed, the sensor is in its closed position as default, with the web application providing the status accordingly. When opening the window, the magnetic sensor opens too and a real-time change of state is sent (Figure 46, Figure 47).

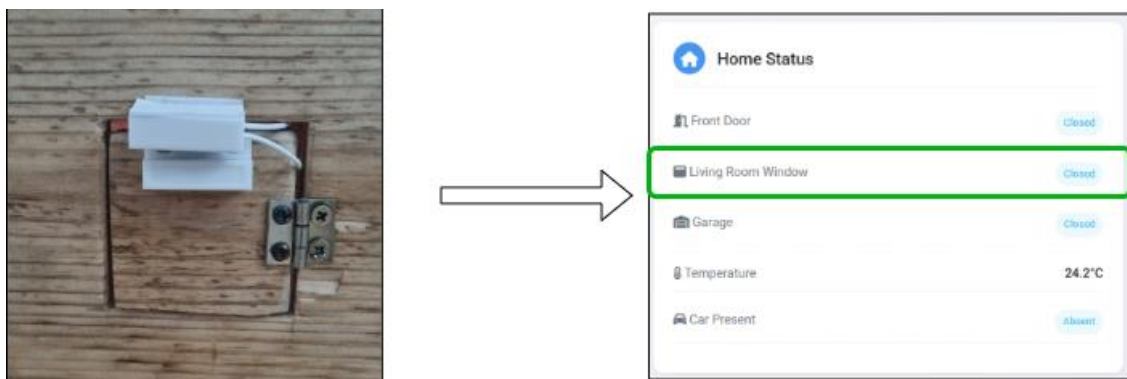


Figure 46 Status update when the window is closed

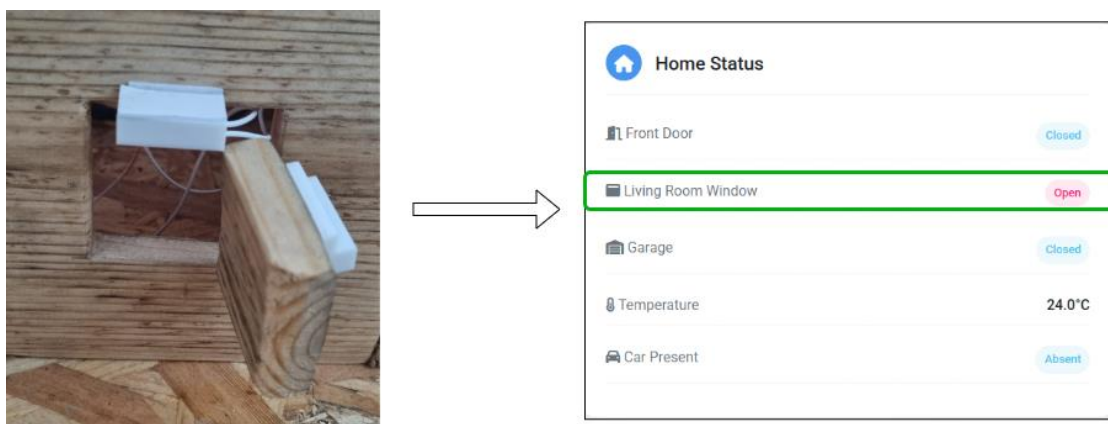


Figure 47 Status update when the window is open

6.2 Front Door Monitoring

The front door status is yet again determined using a magnetic sensor, same as the window status, handled by the same 30-pin ESP32. In the same manner, when the door is closed, the sensor is in its closed position and when opening the door, the magnetic sensor is open and a real-time change of state is sent (Figure 48, Figure 49). However, the difference between the door and the window is the presence of the locking mechanism at the door. As expected, the door can only be opened once the lock is disengaged, operation which is actioned by the “Unlock Door” button, found in the “Security Controls” tab of the web app. For security purposes, the default state of the solenoid lock when the system boots is Locked.

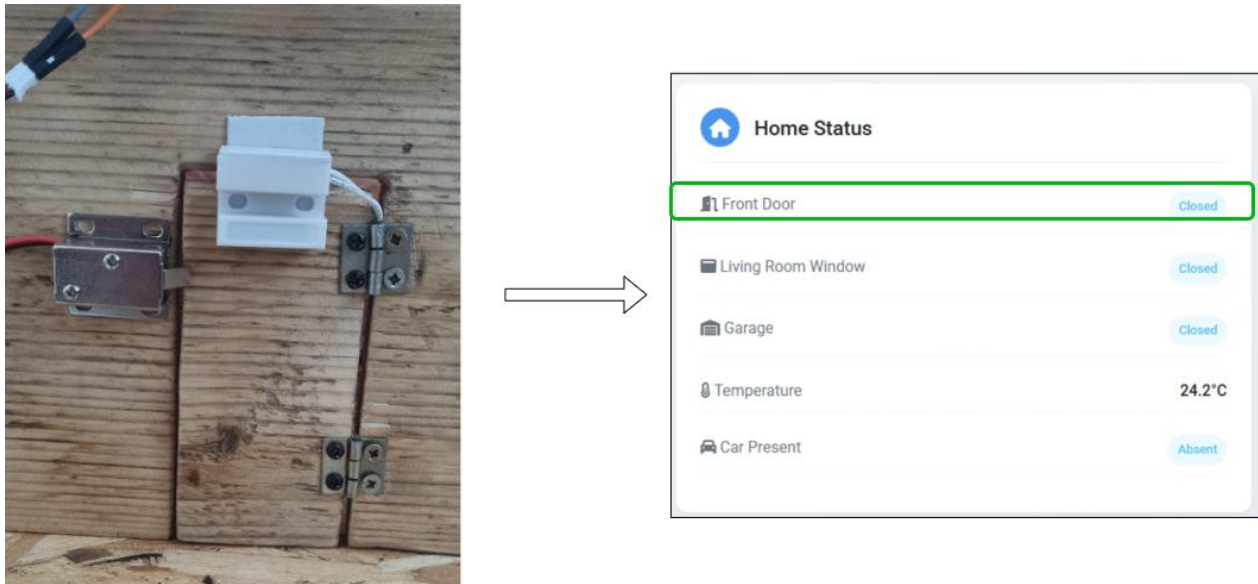


Figure 48 Status update when the door is locked and closed

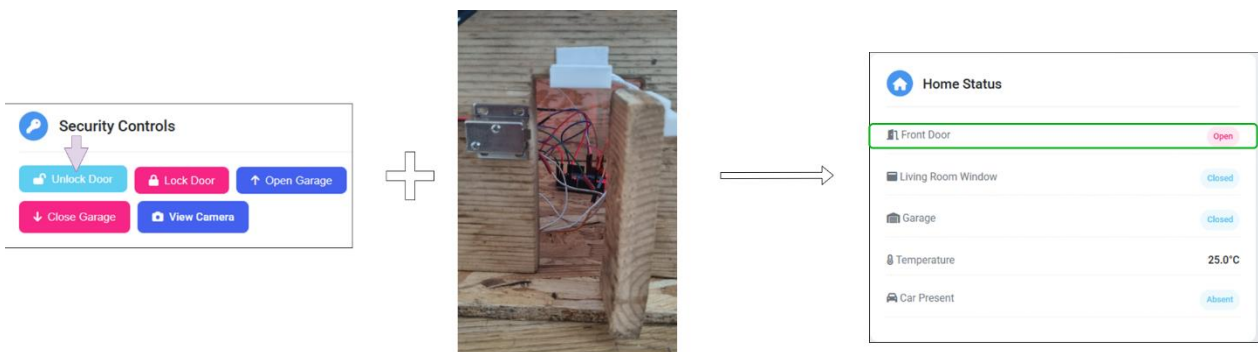


Figure 49 Status update when the door is unlocked and open

6.3 Interior Lighting Control

The interior lighting is represented by an LED controlled by a PWM capable GPIO pin of the 30-pin ESP32. The web app features in the “Lighting Control” tab a slider control allowing for real-time control and fine tuning of the LED. This approach is user and energy-friendly, with a range of 256 intensities to suit any user’s preferences. When the slider is all the way to the left, set on 0, the LED is completely off, with the brightness increasing gradually up to 255 (Figure 50, Figure 51, Figure 52).

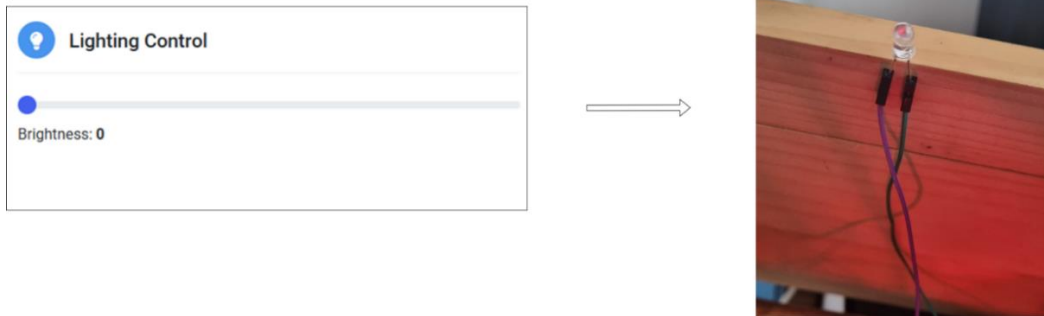


Figure 50 Interior Lighting when brightness is set to 0

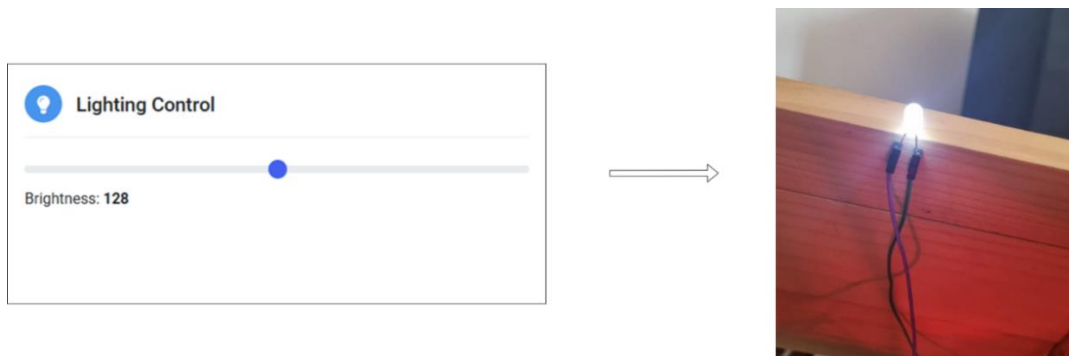


Figure 51 Interior Lighting when brightness is set to 128

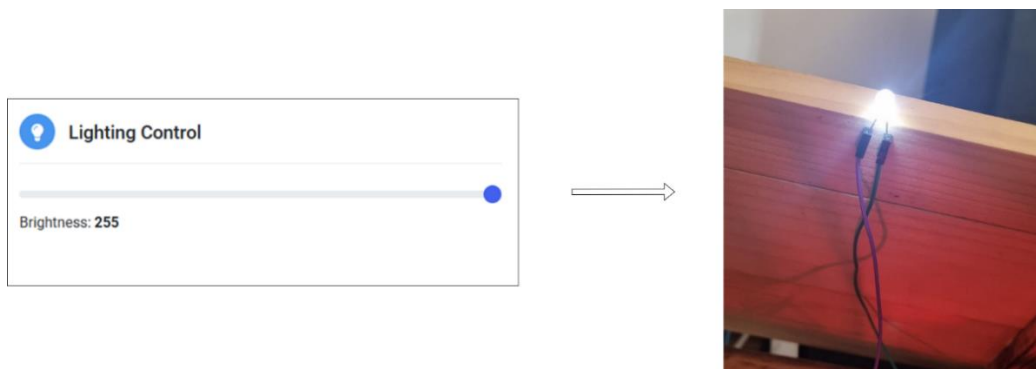


Figure 52 Interior Lighting when brightness is set to 255

6.4 Garage Door Control and Status

The garage door mechanism is a DC motor which has a little shaft attached to it. A thin rope is attached to the shaft with one end, the other being tied to a hook in the lower part of the door. This way, when the motor spins, the rope winds on the shaft, thus lifting the lower part of the door. Due to the hinges being in the upper part of the door, it pivots vertically when it opens.

The web app provides status information and control over the garage door. By default, when the system boots, it is closed. In order to open the door, the “Open Garage” button from the “Security Controls” tab needs to be pressed, this action triggering the status of the door as well. The closing of the garage works the same way, but the “Close Garage” action is followed by an additional check in the browser, ensuring it wasn’t an accidental command. Once the garage closes, the status changes as well (Figure 53, Figure 54).

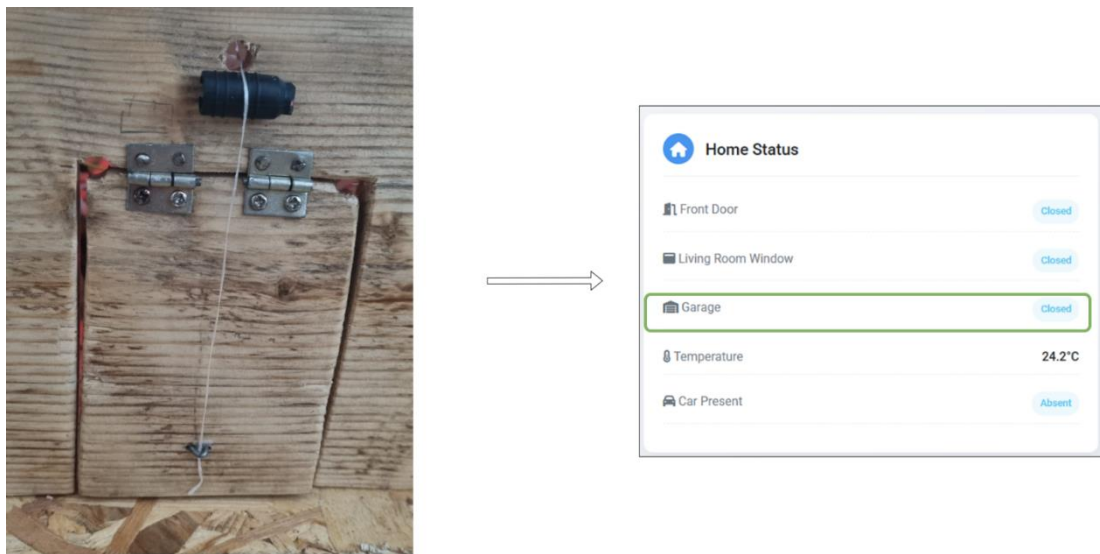


Figure 53 Status when the garage is closed

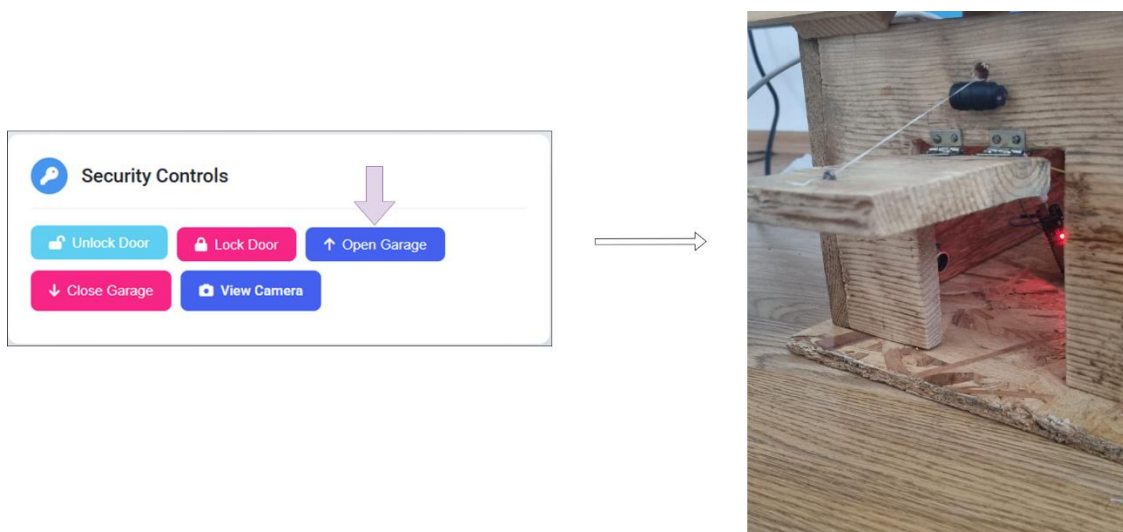


Figure 54 Opening the garage door from the Control section

An additional comfort feature has been added to automate the closing of the door when the user arrives or departs home. The garage door is initially opened from the web app when either arriving or departing home. Next, when the presence status of the car changes, internally, a 10 second countdown is initiated, after which the garage door is closed automatically, with the assumption that the car has been parked or has left the house's perimeter (Figure 55, Figure 56).



Figure 55 Auto-closing the garage feature

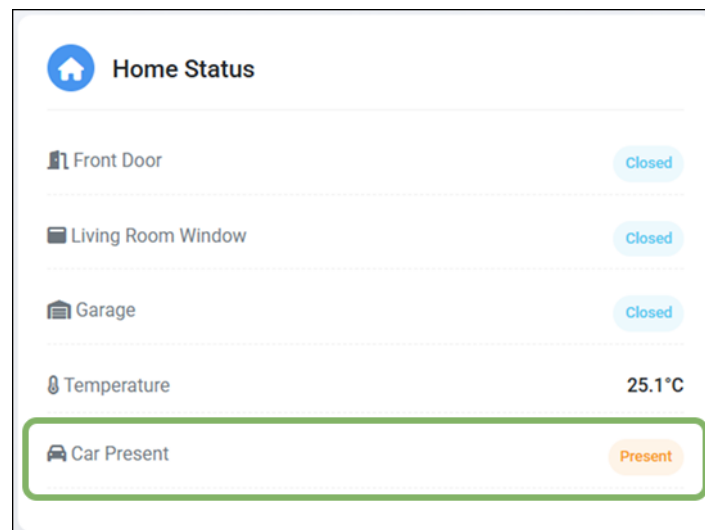


Figure 56 Car status when the car is in the garage

This additional feature has been implemented from a hardware point of view by using an ultrasonic sensor placed near the entrance of the garage and listening and interpreting its readings accordingly.

6.5 Temperature Control

The temperature control has implemented two temperature thresholds, one for the lower limit of the user's comfort zone and one for the upper limit. The default ones are 18°C and 26°C, but the user can change them whenever they feel like from the web application, in the "Climate Control" section. There, the user can enter the desired temperatures and hit the "Save Settings" button in order to confirm the modifications made.

This system simulates a real-life HVAC system as when the ambient temperature provided by the sensor is above the high threshold, a ventilation unit turns on inside the house (Figure 58), and when the ambient temperature is below the lower threshold, a red LED turns on, (Figure 57) simulating a heating unit. When the ambient temperature is between the thresholds, a dead zone was implemented where no unit is on. In no scenario will both the heating and ventilation be on at the same time.

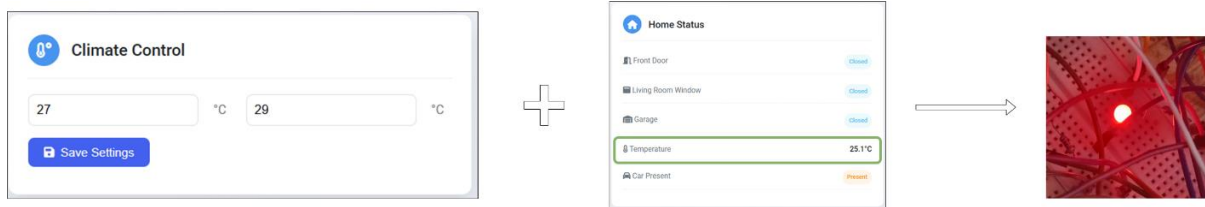


Figure 57 Scenario when heating LED must be on

For temperature thresholds set to 27°C and 29°C, like above in Figure 49, and an ambient temperature of 25.1°C, the expected outcome will be that the heating must turn on, as the ambient temperature is lower than the lower threshold, which is exactly what happens experimentally.

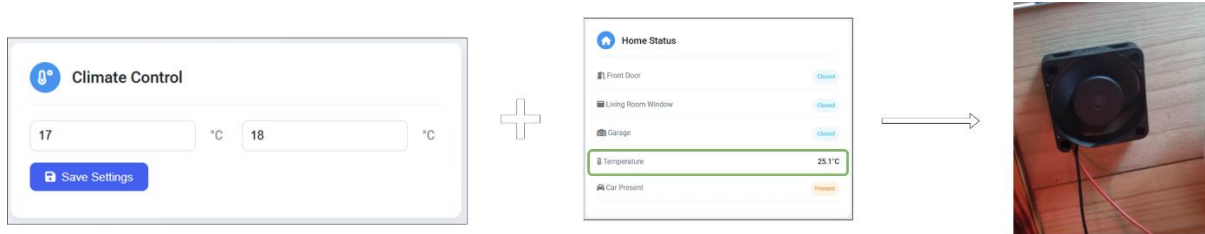


Figure 58 Scenario when the fan must be working

If the temperature thresholds are set lower, like to 17°C and 18°C, as seen above in Figure 50, with the same ambient temperature of 25.1°C, the expected outcome will be that the ventilation must turn on, which does indeed happen, as the ambient temperature is higher than the higher threshold and cooling is needed immediately.

7 Conclusion

This project showcases the successful implementation of a smart home system using development boards to enable real-time status monitoring, automation and control of a diverse range of functions. By combining sensor networks and wireless communication protocols, the system provides a modern and efficient approach to home automation, alongside a user-friendly and intuitive web interface to facilitate user control.

At this stage, the system is implemented with three development boards, each playing a crucial role in creating a well-balanced smart home ecosystem. The Arduino Uno is responsible for the biometric authentication, providing a strong and modern security layer before providing access to the system's entire functionalities. The 30-pin ESP32 acts as the "brain" of the system, the primary control unit, the main handler of all the sensors and actuators. It processes all the data while also taking in requests for automation tasks from the user through the web interface. The ESP32-CAM serves as the web server host, the web interface hub, receiving sensor data and sending requests to the main controller, the ESP32 30-pin via HTTP communication, while also providing the security camera live feed for the user to check at will. This architecture combines efficiency and reliability, combining modern wireless communication with reliable serial communication, all in an ecosystem where each component operates almost independently, yet together they form a single modern and powerful unit.

While being an efficient and stable system, several limitations need consideration. First, the dependence of a single Wi-Fi network and the credentials being hardcoded into the code, introduces a potential point of failure. A potential disruption of connection would disrupt the remote monitoring and control accessibility. Second, the fingerprint authentication, while modern and secure, might raise some security concerns due to a lack of encryption, leaving potential vulnerabilities in the security layer. These constraints highlight key areas for potential improvements in the future.

The project exemplifies the practical implementation of a smart home system using cost-effective components, from sensors to actuators to the development boards, delivering a modern interface with sensor statuses, home automation and security monitoring. While the current implementation seamlessly integrates the hardware and software in order to create a user-friendly smart automation system, it faces challenges in areas such as network reliability and data security and integrity. Future improvements like encryption protocols or mesh networks can tackle and efficiently solve the current limitations of the system.

Artificial intelligence tools were used in this documentation only for auxiliary purposes, such as assisting in rephrasing and translating text for clarity and accessibility. The tools were used in a transparent manner, not influencing the research or development of the project. All ideas and analyses were conducted by the author, maintaining integrity and originality.

8 References

- [1] Stolojescu-Crisan, C., Crisan, C., & Butunoi, B. P. (2021). An IoT-based smart home automation system. *Sensors*, 21(11), 3784.
- [2] Gram-Hanssen, K., & Darby, S. J. (2018). “Home is where the smart is”? Evaluating smart home research and approaches against the concept of home. *Energy Research & Social Science*, 37, 94-101.
- [3] Marikyan, D., Papagiannidis, S., & Alamanos, E. (2019). A systematic review of the smart home literature: A user perspective. *Technological Forecasting and Social Change*, 138, 139-154.
- [4] Sovacool, B. K., & Del Rio, D. D. F. (2020). Smart home technologies in Europe: A critical review of concepts, benefits, risks and policies. *Renewable and sustainable energy reviews*, 120, 109663.
- [5] Nicholls, L., Strengers, Y., & Sadowski, J. (2020). Social impacts and control in the smart home. *Nature Energy*, 5(3), 180-182.
- [6] “ESP32”, Available: <https://en.wikipedia.org/wiki/ESP32>, Accessed May 2025
- [7] “ESP32-S Series”, Available: <https://www.espressif.com/en/products/modules/esp32-cam>, Accessed May 2025
- [8] “All about ESP32 Camera Module”, Available: <https://robocraze.com/blogs/post/all-about-esp32-camera-module?srltid=AfmBOoqfutrTVzyarjdIcuJG2c1zEi5D5IoGnEqMxEcD2pzaIMk2IN8W>, Accessed May 2025
- [9] “Arduino Uno R3”, Available: <https://docs.arduino.cc/hardware/uno-rev3/>, Accessed May 2025
- [10] “What is Arduino Uno?”, Available: <https://robu.in/what-is-arduino-uno/>, Accessed May 2025
- [11] “Arduino Uno”, Available: https://en.wikipedia.org/wiki/Arduino_Uno, Accessed May 2025
- [12] https://www.optimusdigital.ro/ro/placi-cu-bluetooth/4371-placa-de-dezvoltare-esp32-cu-wifi-i-bluetooth-42.html?gad_source=1&gad_campaignid=19615979487&gbraid=0AAAAADv-p3BSNtw6B-MAPx_e9LE-9Juwr&gclid=CjwKCAjwi-DBBhA5EiwAXOHsGWhAweBzzmZLZsS8ibkIbLjcW1DO_v-IBLdNTxRv_PlbmCwmi8cLPhoCAXcQAvD_BwE
- [13] <https://www.bitmi.ro/placa-de-dezvoltare-esp32-cam-conexiune-wifi-si-bluetooth-ov2640-2mp-10990.html?2pau=bafcd4c40&2ptt=quicklink&2ptu=989f060e9&2pdlst=CjwKCAjwi->

[DBBhA5EiwAXOHsGczrxYTTHmwIAHk3B3k0tXKRCtoqZvfLZfzUWQiRaU3mMAK_k
e5bVhoCRWsQAvD_BwE&gad_source=1&gad_campaignid=22602772467&gbraid=0AAA
AA_k3_EfCS1dVgP-ns43m6qZEIcpWS&gclid=CjwKCAjwi-
DBBhA5EiwAXOHsGczrxYTTHmwIAHk3B3k0tXKRCtoqZvfLZfzUWQiRaU3mMAK_k
e5bVhoCRWsQAvD_BwE](https://store.arduino.cc/products/arduino-uno-rev3?srltid=AfmBOoqoV7_AuEuwiVN9ieVqE7XhrdDxSBwqfryzOC_Qh9OLqH_Hffnd)

[14] [https://store.arduino.cc/products/arduino-uno-
rev3?srltid=AfmBOoqoV7_AuEuwiVN9ieVqE7XhrdDxSBwqfryzOC_Qh9OLqH_Hffnd](https://store.arduino.cc/products/arduino-uno-rev3?srltid=AfmBOoqoV7_AuEuwiVN9ieVqE7XhrdDxSBwqfryzOC_Qh9OLqH_Hffnd)

[15] “DHT-11 Temperature and Humidity Sensor”, Available: <https://components101.com/sensors/dht11-temperature-sensor>, Accessed June 2025

[16] [https://www.bitmi.ro/modul-senzor-de-umiditate-si-temperatura-dht11-ky-015-
10637.html?gad_source=1&gad_campaignid=22005721655&gbraid=0AAAAADLag-
mtVVjPmg3lInFJ_eS9JpdYe&gclid=CjwKCAjwruXBBhArEiwACBRtHQQR3o4XZunmsd
gAAF0TyEuJH-FW_A9pTI6lyPRW_Nfx51nn4hqeRoCc18QAvD_BwE](https://www.bitmi.ro/modul-senzor-de-umiditate-si-temperatura-dht11-ky-015-10637.html?gad_source=1&gad_campaignid=22005721655&gbraid=0AAAAADLag-mtVVjPmg3lInFJ_eS9JpdYe&gclid=CjwKCAjwruXBBhArEiwACBRtHQQR3o4XZunmsd_gAAF0TyEuJH-FW_A9pTI6lyPRW_Nfx51nn4hqeRoCc18QAvD_BwE)

[17] “Adafruit Optical Fingerprint Sensor”, Available: [https://learn.adafruit.com/adafruit-
optical-fingerprint-sensor](https://learn.adafruit.com/adafruit-optical-fingerprint-sensor), Accessed June 2025

[18] “Guidance of Fingerprint Sensor Module”, Available: <https://www.supremainc.com/embedded-modules/en/modules/fingerprint-module.asp>,
Accessed June 2025

[19] [https://www.bitmi.ro/electronica/modul-senzor-optic-de-amprente-compatibil-arduino-
10450.html](https://www.bitmi.ro/electronica/modul-senzor-optic-de-amprente-compatibil-arduino-10450.html)

[20] “DC Motor Drives”, Available: <https://www.electrical4u.com/dc-motor-drives/>,
Accessed June 2025

[21] “What is a motor driver?”, Available: [https://www.campuscomponent.com/blogs/post/What-is-motor-
driver?srltid=AfmBOoovqrwkbrij8bsDIZi5jYIAf7J7NVFxAALW-moWXQthhQhKxJyWil](https://www.campuscomponent.com/blogs/post/What-is-motor-driver?srltid=AfmBOoovqrwkbrij8bsDIZi5jYIAf7J7NVFxAALW-moWXQthhQhKxJyWil),
Accessed June 2025

[22] [https://ardushop.ro/ro/electronica/752-motor-dc-3v-6v-cu-reductor-1-48-
6427854009609.html?gad_source=1&gad_campaignid=22058879462&gbraid=0AAAAAD1
KU-4LI9_Rh9WI-
VlfmfVgGHi1d&gclid=CjwKCAjwruXBBhArEiwACBRtHf14T1e1DIEkr92IxbEAd-fd-
GSdwygF6t48-Y2nnGVQBpFojV2xwhoCQagQAvD_BwE](https://ardushop.ro/ro/electronica/752-motor-dc-3v-6v-cu-reductor-1-48-6427854009609.html?gad_source=1&gad_campaignid=22058879462&gbraid=0AAAAAD1KU-4LI9_Rh9WI-VlfmfVgGHi1d&gclid=CjwKCAjwruXBBhArEiwACBRtHf14T1e1DIEkr92IxbEAd-fd-GSdwygF6t48-Y2nnGVQBpFojV2xwhoCQagQAvD_BwE)

[23] “How electromagnets work”, Available: <https://science.howstuffworks.com/electromagnet.htm>, Accessed June 2025

[24] “Arduino – Solenoid Lock”, Available: [https://arduinogetstarted.com/tutorials/arduino-
solenoid-lock](https://arduinogetstarted.com/tutorials/arduino-solenoid-lock), Accessed June 2025

[25] <https://www.takigen.com/products/list/L-002>

[26] “What is a Magnetic Switch and What is it Used For?”, Available: [https://eltra-
trade.com/blog/what-is-a-magnetic-switch-and-what-is-it-used-for](https://eltrade.com/blog/what-is-a-magnetic-switch-and-what-is-it-used-for), Accessed June 2025

[27] https://www.a2t.ro/default-category/contact-magnetic-aplicat-alb-nc-autoadeziv?gad_source=1&gad_campaignid=18568628397&gbraid=0AAAAADk_d8k10lMQ_k_7i3zSDPp1zNJHg&gclid=CjwKCAjwruXBBhArEiwACBRtHfo_p1X4aLjCapOB8rxML1Mxh2JqGs1qCt6hr38wux8Twk2RzngcPxoCUQUQAvD_BwE

[28] “Ultrasonic Transducer”, Available: https://en.wikipedia.org/wiki/Ultrasonic_transducer, Accessed June 2025

[29] “How Ultrasonic Sensors Work?”, Available: <https://maxbotix.com/blogs/blog/how-ultrasonic-sensors-work?srsltid=AfmBOopvu50qdQiAAoYlsA0ih0-Ans1D-NMuURYfoGdlycH2QTRzzHzp>, Accessed June 2025

[30] <https://contactelectric.ro/senzori-arduino/2726-senzor-ultrasonic-hc-sr04p-oky3261-2?gQT=1>

[31] “Relay”, Available: <https://en.wikipedia.org/wiki/Relay>, Accessed June 2025

[32] “How to use a relay module?”, Available: <https://lab.arts.ac.uk/books/physical-computing/page/how-to-use-a-relay-module>, Accessed June 2025

[33] “What is a motor driver?”, Available: https://robocraze.com/blogs/post/what-is-motor-driver?srsltid=AfmBOor2jKJLg4oNHdDoAqBE_ww2AaBhXA4fA5iNGzm5AOVxWHj2PHKZ, Accessed June 2025

[34] “DC Fan and its working principle”, Available: <https://www.leipole.net/article/dc-fan-and-its-working-principle.html#:~:text=In%20simple%20terms%2C%20a%20DC,that%20the%20fan%20blades%20rotate>, Accessed June 2025

[35] “ESP32 Pinout Reference”, Available: <https://lastminuteengineers.com/esp32-pinout-reference/>, Accessed June 2025

[36] “PUT vs POST – Comparing HTTP Methods”, Available: <https://www.keycdn.com/support/put-vs-post>, Accessed June 2025

[37] “What is a Smart Home?”, Available: <https://builtin.com/articles/what-is-a-smart-home>, Accessed June 2025

[38] “US Smart Home Statistics (2019-2028)”, Available: <https://www.oberlo.com/statistics/smart-home-statistics>, Accessed June 2025

[39] “ESP32: ESP-NOW Web Server Sensor Dashboard (ESP-NOW + Wi-Fi)”, Available: <https://randomnerdtutorials.com/esp32-esp-now-wi-fi-web-server/>, Accessed June 2025

[40] “Smart Home: Threats and Countermeasures”, Available: <https://www.rambus.com/iot/smart-home/>, Accessed June 2025

[41] “6 Smart Home Security Risks – And How to Protect Yourself”, Available: <https://www.zolo.ca/blog/smart-home-security-risks>, Accessed June 2025

[42] “ESP32 WiFi Tutorial & Library Examples (Arduino IDE)”, Available: <https://deepbluembedded.com/esp32-wifi-library-examples-tutorial-arduino/>, Accessed June 2025

[43] <https://www.eon.ro/statie-energie/ghid-energie/ce-este-tehnologia-smart-home-si-la-ce-ne-ajuta>

9 Annexes

uno

```
#include <SoftwareSerial.h>
#include <Adafruit_Fingerprint.h>

SoftwareSerial mySerial(4, 5); // RX, TX for fingerprint sensor
Adafruit_Fingerprint finger(&mySerial);

void setup() {
  Serial.begin(9600);
  mySerial.begin(57600);
  finger.begin(57600);

  delay(1000);

  if (finger.verifyPassword()) {
    Serial.println("Fingerprint sensor found!");
  } else {
    Serial.println("Fingerprint sensor NOT found!");
    while (1) delay(100); // halt if sensor not found
  }
}

void loop() {
  uint8_t p = finger.getImage();

  if (p == FINGERPRINT_NOFINGER) {
    // No finger detected, do nothing, no spam
    delay(100);
    return;
  }

  if (p != FINGERPRINT_OK) {
    Serial.println("Error reading finger image");
    sendAccessStatus(false);
    delay(1000);
    return;
  }

  p = finger.image2Tz();
  if (p != FINGERPRINT_OK) {
    Serial.println("Failed to convert image");
    sendAccessStatus(false);
  }
}
```

```

    delay(1000);
    return;
}

p = finger.fingerSearch();
if (p == FINGERPRINT_OK) {
    Serial.print("Fingerprint ID found: ");
    Serial.println(finger.fingerID);
    sendAccessStatus(true);
} else if (p == FINGERPRINT_NOTFOUND) {
    Serial.println("Fingerprint not recognized");
    sendAccessStatus(false);
} else {
    Serial.print("Fingerprint search error: ");
    Serial.println(p);
    sendAccessStatus(false);
}

delay(1000); // wait a second before next scan to avoid spamming
}

void sendAccessStatus(bool granted) {
    if (granted) {
        Serial.println("GRANTED");
    } else {
        Serial.println("DENIED");
    }
}
}

```

```

#include <WiFi.h>
#include <HTTPClient.h>
#include <DHT.h>
#include <WebServer.h>
#include <Preferences.h>

WebServer localServer(80);
Preferences preferences;

// Pin Definitions
#define GARAGE_MOTOR_PIN_1 14
#define GARAGE_MOTOR_PIN_2 27
#define ULTRASONIC_TRIG 13
#define ULTRASONIC_ECHO 12
#define DOOR_REED_PIN 32
#define WINDOW_REED_PIN 33
#define DHTPIN 25
#define DHTTYPE DHT11
#define COOLING_LED 26
#define HEATING_LED 15
#define SOLENOID_PIN 4
#define PWM_LED_PIN 21

// WiFi and Server
const char* ssid = "Licenta";
const char* password = "123456789";
const char* esp32CamIP = "192.168.94.55";

// Add these constants near the top of your file
const unsigned long SEND_RETRY_INTERVAL = 50; // ms between HTTP attempts
const int PWM_CHANGE_THRESHOLD = 5; // Min change to send update

// Auto close variables
bool autoCloseTriggered = false;
unsigned long autoCloseStartTime = 0;
const unsigned long AUTO_CLOSE_DELAY = 10000; // 10 seconds
bool lastCarPresent = false;

```



```

// System State
float currentTemp = 0.0;
int coolingThreshold;
int heatingThreshold;
bool garageOpen = false;
bool accessGranted = false;
bool carPresent = false;
bool doorOpen = false;
bool windowOpen = false;
int pwmValue;
const unsigned long sensorUpdateInterval = 1000; // Update sensors every 1000ms (1 second)

DHT dht(DHTPIN, DHTTYPE);

void setup() {
  Serial.begin(115200);
  Serial2.begin(9600, SERIAL_8N1, 16, 17);

  // Initialize preferences
  preferences.begin("smart-home", false);
  pwmValue = preferences.getInt("pwm", 128);
  coolingThreshold = preferences.getInt("coolThresh", 27);
  heatingThreshold = preferences.getInt("heatThresh", 22);
  accessGranted = false;
  preferences.putBool("access", false); // Reset on every startup

  // Initialize pins
  pinMode(GARAGE_MOTOR_PIN_1, OUTPUT);
  pinMode(GARAGE_MOTOR_PIN_2, OUTPUT);
  pinMode(COOLING_LED, OUTPUT);
  digitalWrite(COOLING_LED, HIGH); // relay OFF by default
  pinMode(HEATING_LED, OUTPUT);
  pinMode(SOLENOID_PIN, OUTPUT);
  digitalWrite(SOLENOID_PIN, accessGranted ? LOW : HIGH);

  pinMode(DOOR_REED_PIN, INPUT_PULLUP);
  pinMode(WINDOW_REED_PIN, INPUT_PULLUP);
  pinMode(ULTRASONIC_TRIG, OUTPUT);
  pinMode(ULTRASONIC_ECHO, INPUT);

```

```

ledcAttach(PWM_LED_PIN, 5000, 8);
ledcWrite(PWM_LED_PIN, pwmValue);

dht.begin();

// Connect to WiFi
WiFi.begin(ssid, password);
while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
}
Serial.println("\nWiFi connected");
Serial.print("IP Address: ");
Serial.println(WiFi.localIP()); // This line shows the IP address
notifyCamAccess(false);

localServer.on("/cmd", HTTP_GET, []() {
    if (localServer.hasArg("cool") && localServer.hasArg("heat")) {
        coolingThreshold = localServer.arg("cool").toInt();
        heatingThreshold = localServer.arg("heat").toInt();
        preferences.putInt("coolThresh", coolingThreshold);
        preferences.putInt("heatThresh", heatingThreshold);
        Serial.println("Received thresholds - Cooling: " + String(coolingThreshold) + ", Heating: " + String(heatingThreshold));
    }

    if (localServer.hasArg("light")) {
        pwmValue = localServer.arg("light").toInt();
        preferences.putInt("pwm", pwmValue);
        setPWM(pwmValue);
        Serial.println("Received light PWM value: " + String(pwmValue));
    }

    if (localServer.hasArg("lock")) {
        Serial.println("Received LOCK command for door solenoid");
        setSolenoid(true); // Lock the door
    }

    if (localServer.hasArg("unlock")) {
        Serial.println("Received UNLOCK command for door solenoid");
        setSolenoid(false); // Unlock the door
    }
}

```

```

    if (localServer.hasArg("openGarage")) {
        Serial.println("Received OPEN GARAGE command");
        openGarage();
    }

    if (localServer.hasArg("closeGarage")) {
        Serial.println("Received CLOSE GARAGE command");
        closeGarage();
    }

    localServer.send(200, "text/plain", "OK");
});
localServer.begin();
}
// [Rest of your existing functions remain unchanged]
// (keep all your existing loop(), updateSensors(), handleTemperature(), etc.)

void loop() {
    checkSerialFingerprint();
    localServer.handleClient();
    updateSensors();
    if (autoCloseTriggered && (millis() - autoCloseStartTime >= AUTO_CLOSE_DELAY)) {
        Serial.println("Auto-close timer expired. Closing garage.");
        closeGarage();
        autoCloseTriggered = false;
    }
}

void updateSensors() {
    static unsigned long lastUpdate = 0;
    if (millis() - lastUpdate < sensorUpdateInterval) return;
    lastUpdate = millis();

    if (accessGranted) {
        handleTemperature();
        handleUltrasonicGarage();
    }

    doorOpen = (digitalRead(DOOR_REED_PIN) == HIGH);
    windowOpen = (digitalRead(WINDOW_REED_PIN) == HIGH);
}

```

```

    sendSensorDataToCam();
}

void handleTemperature() {
    float temp = dht.readTemperature();
    if (!isnan(temp)) {
        currentTemp = temp;
        digitalWrite(COOLING_LED, currentTemp >= coolingThreshold ? LOW : HIGH);
        digitalWrite(HEATING_LED, currentTemp <= heatingThreshold ? HIGH : LOW);
    }
}

void handleUltrasonicGarage() {
    digitalWrite(ULTRASONIC_TRIG, LOW);
    delayMicroseconds(2);
    digitalWrite(ULTRASONIC_TRIG, HIGH);
    delayMicroseconds(10);
    digitalWrite(ULTRASONIC_TRIG, LOW);

    long duration = pulseIn(ULTRASONIC_ECHO, HIGH, 30000);
    int distance = duration * 0.034 / 2;

    bool previousCarPresent = carPresent;
    carPresent = (distance > 0 && distance < 15);

    // If car presence changed and garage is open, start auto-close timer
    if (garageOpen && carPresent != previousCarPresent && !autoCloseTriggered) {
        Serial.println("Car presence changed while garage is open. Starting auto-close countdown.");
        autoCloseStartTime = millis();
        autoCloseTriggered = true;
    }
}

void openGarage() {
    Serial.println("Opening garage door...");
    digitalWrite(GARAGE_MOTOR_PIN_1, HIGH);
    digitalWrite(GARAGE_MOTOR_PIN_2, LOW);
    delay(1500); // Adjust this delay based on your garage door operation time
    digitalWrite(GARAGE_MOTOR_PIN_1, LOW);
    digitalWrite(GARAGE_MOTOR_PIN_2, LOW);
}

```

```

    garageOpen = true;
    Serial.println("Garage door opened");
}

void closeGarage() {
    Serial.println("Closing garage door...");
    digitalWrite(GARAGE_MOTOR_PIN_1, LOW);
    digitalWrite(GARAGE_MOTOR_PIN_2, HIGH);
    delay(1450); // Adjust this delay based on your garage door operation time
    digitalWrite(GARAGE_MOTOR_PIN_1, LOW);
    digitalWrite(GARAGE_MOTOR_PIN_2, LOW);
    garageOpen = false;
    Serial.println("Garage door closed");
    autoCloseTriggered = false;
}

void checkSerialFingerprint() {
    while (Serial2.available()) {
        String msg = Serial2.readStringUntil('\n');
        msg.trim();
        if (msg == "GRANTED" && !accessGranted) {
            accessGranted = true;
            setSolenoid(true);
            notifyCamAccess(true);
        } else if (msg == "DENIED") {
            Serial.println("Access denied. Ignoring...");
            // Do NOT revoke access.
        }
    }
}

void notifyCamAccess(bool access) {
    HTTPClient http;
    http.begin("http://" + String(esp32CamIP) + "/auth?access=" + (access ? "1" : "0"));
    http.GET();
    http.end();
}

```

```

void sendDataToCam() {
    HTTPClient http;
    String url = "http://" + String(esp32CamIP) + "/update?";
    url += "door=" + String(doorOpen ? "open" : "closed");
    url += "&window=" + String(windowOpen ? "open" : "closed");
    url += "&garage=" + String(garageOpen ? "open" : "closed");
    url += "&temp=" + (isnan(currentTemp) ? "nan" : String(currentTemp, 1));
    url += "&car=" + String(carPresent ? "1" : "0");
    url += "&pwm=" + String(pwmValue); // Add current PWM value

    http.begin(url);
    http.setTimeout(200); // Reduce timeout
    http.addHeader("Cache-Control", "no-cache");
    http.GET();
    http.end();
}

void setPWM(int value) {
    pwmValue = constrain(value, 0, 255); // Ensure value is within valid range
    preferences.putInt("pwm", pwmValue); // Save to preferences
    ledcWrite(PWM_LED_PIN, pwmValue);
    Serial.println("Setting PWM to: " + String(pwmValue));
    sendDataToCam();
}

void setSolenoid(bool unlocked) {
    digitalWrite(SOLENOID_PIN, unlocked ? HIGH : LOW);
}

void setThresholds(int cooling, int heating) {
    coolingThreshold = cooling;
    heatingThreshold = heating;
}

```

```

#include "esp_camera.h"
#include <WiFi.h>
#include <WebServer.h>
#include <HTTPClient.h>
#include <Preferences.h>

#define CAMERA_MODEL_AI_THINKER
#include "camera_pins.h"

void handleRoot();
void handleAuth();
void handleUpdate();
void handleLight();
void handleThresholds();
void handleCameraOn();
void handleCameraOff();
void handleStatus();
void handleStream();
void sendCommandToESP32(const String& path);

const char* ssid = "Licenta";
const char* password = "123456789";
const char* esp32Ip = "192.168.94.82";

WebServer server(80);
Preferences preferences;

// System State
bool accessGranted = false;
bool cameraActive = false;
String doorStatus = "unknown";
String windowStatus = "unknown";
String garageStatus = "unknown";
String temperature = "unknown";
String carPresent = "0";
int pwmValue;
int tempLow;
int tempHigh;

```



```

void setup() {
    Serial.begin(115200);

    // Initialize preferences
    preferences.begin("smart-home", false);
    pwmValue = preferences.getInt("pwm", 128);
    tempLow = preferences.getInt("tempLow", 18);
    tempHigh = preferences.getInt("tempHigh", 26);
    accessGranted = false;
    preferences.putBool("access", false);

    // Camera config
    camera_config_t config;
    config.ledc_channel = LEDC_CHANNEL_0;
    config.ledc_timer = LEDC_TIMER_0;
    config.pin_d0 = Y2_GPIO_NUM;
    config.pin_d1 = Y3_GPIO_NUM;
    config.pin_d2 = Y4_GPIO_NUM;
    config.pin_d3 = Y5_GPIO_NUM;
    config.pin_d4 = Y6_GPIO_NUM;
    config.pin_d5 = Y7_GPIO_NUM;
    config.pin_d6 = Y8_GPIO_NUM;
    config.pin_d7 = Y9_GPIO_NUM;
    config.pin_xclk = XCLK_GPIO_NUM;
    config.pin_pclk = PCLK_GPIO_NUM;
    config.pin_vsync = VSYNC_GPIO_NUM;
    config.pin_href = HREF_GPIO_NUM;
    config.pin_sccb_sda = SIOD_GPIO_NUM;
    config.pin_sccb_scl = SIOC_GPIO_NUM;
    config.pin_pwdn = PWDN_GPIO_NUM;
    config.pin_reset = RESET_GPIO_NUM;
    config.xclk_freq_hz = 20000000;
    config.pixel_format = PIXFORMAT_JPEG;
    config.frame_size = FRAMESIZE_VGA;
    config.fb_location = CAMERA_FB_IN_PSRAM;
    config.jpeg_quality = 12;
    config.fb_count = 1;

```

```

// Camera init
esp_err_t err = esp_camera_init(&config);
if (err != ESP_OK) {
    Serial.printf("Camera init failed with error 0x%x", err);
    return;
}

// Connect to WiFi
WiFi.begin(ssid, password);
while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
}
Serial.println("");
Serial.println("WiFi connected");
Serial.print("IP Address: ");
Serial.println(WiFi.localIP()); // This line shows the IP address
// Server routes
// Server routes
server.on("/", HTTP_GET, handleRoot);
server.on("/auth", HTTP_GET, handleAuth);
server.on("/update", HTTP_GET, handleUpdate);
server.on("/lock", HTTP_GET, []() {
    Serial.println("Sending LOCK command to main ESP32");
    sendCommandToESP32("/cmd?lock=1");
    server.send(200, "text/plain", "OK");
});

server.on("/unlock", HTTP_GET, []() {
    Serial.println("Sending UNLOCK command to main ESP32");
    sendCommandToESP32("/cmd?unlock=1");
    server.send(200, "text/plain", "OK");
});

server.on("/openGarage", HTTP_GET, []() {
    Serial.println("Sending OPEN GARAGE command to main ESP32");
    sendCommandToESP32("/cmd?openGarage=1");
    server.send(200, "text/plain", "OK");
});

```

```

server.on("/closeGarage", HTTP_GET, []() {
    Serial.println("Sending CLOSE GARAGE command to main ESP32");
    sendCommandToESP32("/cmd?closeGarage=1");
    server.send(200, "text/plain", "OK");
});

server.on("/light", HTTP_GET, handleLight);
server.on("/thresholds", HTTP_GET, handleThresholds);
server.on("/camera/on", HTTP_GET, handleCameraOn);
server.on("/camera/off", HTTP_GET, handleCameraOff);
server.on("/status", HTTP_GET, handleStatus);
server.on("/stream", HTTP_GET, handleStream);

server.begin();
Serial.println("HTTP server started");
}

void loop() {
    server.handleClient();
}

void handleRoot() {
    String html = R"rawliteral(
<!DOCTYPE html><html><head>
<title>Smart Home Hub</title>
<meta name="viewport" content="width=device-width, initial-scale=1">
<link href="https://fonts.googleapis.com/css2?family=Roboto:wght@300;400;500;700&display=swap" rel="stylesheet">
<style>
:root {
  --primary: #4361ee;
  --secondary: #3f37c9;
  --accent: #4895ef;
  --danger: #f72585;
  --success: #4cc9f0;
  --warning: #f8961e;
  --dark: #212529;
  --light: #f8f9fa;
  --gray: #6c757d;
}
)rawliteral";
}

```

```

* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

body {
  font-family: 'Roboto', sans-serif;
  background: linear-gradient(135deg, #f5f7fa 0%, #e4e8f0 100%);
  color: var(--dark);
  line-height: 1.6;
  padding: 20px;
  min-height: 100vh;
}

.container {
  max-width: 1200px;
  margin: 0 auto;
}

header {
  text-align: center;
  margin-bottom: 30px;
  animation: fadeIn 0.8s ease-out;
}

h1 {
  font-size: 2.5rem;
  color: var(--primary);
  margin-bottom: 10px;
  font-weight: 700;
}

.subtitle {
  color: var(--gray);
  font-size: 1.1rem;
}

```

```

.grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(300px, 1fr));
  gap: 20px;
  margin-bottom: 30px;
}

.card {
  background: white;
  border-radius: 12px;
  padding: 25px;
  box-shadow: 0 10px 20px rgba(0,0,0,0.05);
  transition: transform 0.3s ease, box-shadow 0.3s ease;
}

.card:hover {
  transform: translateY(-5px);
  box-shadow: 0 15px 30px rgba(0,0,0,0.1);
}

.card-header {
  display: flex;
  align-items: center;
  margin-bottom: 20px;
  padding-bottom: 15px;
  border-bottom: 1px solid rgba(0,0,0,0.05);
}

.card-icon {
  width: 40px;
  height: 40px;
  border-radius: 50%;
  background: var(--accent);
  display: flex;
  align-items: center;
  justify-content: center;
  margin-right: 15px;
  color: white;
  font-size: 1.2rem;
}

```

```

.card-title {
  font-size: 1.3rem;
  font-weight: 500;
  color: var(--dark);
}

.status-item {
  display: flex;
  justify-content: space-between;
  margin-bottom: 12px;
  padding: 10px 0;
  border-bottom: 1px dashed rgba(0,0,0,0.05);
}

.status-label {
  font-weight: 400;
  color: var(--gray);
}

.status-value {
  font-weight: 500;
}

.btn {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  padding: 10px 20px;
  border-radius: 8px;
  font-weight: 500;
  cursor: pointer;
  transition: all 0.3s ease;
  border: none;
  text-decoration: none;
  font-size: 0.95rem;
  margin-right: 10px;
  margin-bottom: 10px;
}

```

```

.btn i {
  margin-right: 8px;
}

.btn-primary {
  background: var(--primary);
  color: white;
}

.btn-primary:hover {
  background: var(--secondary);
  transform: translateY(-2px);
}

.btn-danger {
  background: var(--danger);
  color: white;
}

.btn-danger:hover {
  opacity: 0.9;
  transform: translateY(-2px);
}

.btn-success {
  background: var(--success);
  color: white;
}

.btn-success:hover {
  opacity: 0.9;
  transform: translateY(-2px);
}

.btn-group {
  display: flex;
  flex-wrap: wrap;
  margin-top: 15px;
}

```

```

.slider-container {
  margin-top: 15px;
}

.slider {
  -webkit-appearance: none;
  width: 100%;
  height: 8px;
  border-radius: 4px;
  background: #e9ecef;
  outline: none;
  margin-bottom: 15px;
}

.slider::-webkit-slider-thumb {
  -webkit-appearance: none;
  appearance: none;
  width: 20px;
  height: 20px;
  border-radius: 50%;
  background: var(--primary);
  cursor: pointer;
  transition: all 0.2s ease;
}

.slider::-webkit-slider-thumb:hover {
  transform: scale(1.1);
}

.temp-control {
  display: flex;
  gap: 15px;
  margin-top: 15px;
}

```



```

.temp-input {
  flex: 1;
  padding: 10px;
  border: 1px solid #dee2e6;
  border-radius: 8px;
  font-size: 1rem;
}

#denied {
  text-align: center;
  padding: 50px 20px;
  background: white;
  border-radius: 12px;
  box-shadow: 0 10px 20px rgba(0,0,0,0.05);
  max-width: 500px;
  margin: 0 auto;
  animation: fadeIn 0.8s ease-out;
}

#denied h2 {
  color: var(--danger);
  margin-bottom: 15px;
}

#denied p {
  color: var(--gray);
  margin-bottom: 20px;
}

#denied .icon {
  font-size: 3rem;
  color: var(--danger);
  margin-bottom: 20px;
}

#granted {
  display: none;
  animation: fadeIn 0.8s ease-out;
}

```

```

.badge {
  display: inline-block;
  padding: 5px 10px;
  border-radius: 20px;
  font-size: 0.8rem;
  font-weight: 500;
}

.badge-success {
  background: rgba(76, 201, 240, 0.1);
  color: var(--success);
}

.badge-danger {
  background: rgba(247, 37, 133, 0.1);
  color: var(--danger);
}

.badge-warning {
  background: rgba(248, 150, 30, 0.1);
  color: var(--warning);
}

@keyframes fadeIn {
  from { opacity: 0; transform: translateY(20px); }
  to { opacity: 1; transform: translateY(0); }
}

@media (max-width: 768px) {
  .grid {
    grid-template-columns: 1fr;
  }

  .btn-group {
    justify-content: center;
  }

  .temp-control {
    flex-direction: column;
  }
}

```

```

</style>
<link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.0.0-beta3/css/all.min.css">
</head>
<body>
  <div class="container">
    <div id="denied">
      <div class="icon"><i class="fas fa-fingerprint"></i></div>
      <h2>Access Required</h2>
      <p>Please authenticate using your fingerprint to access the Smart Home controls</p>
      <div class="spinner"><i class="fas fa-spinner fa-spin"></i></div>
    </div>

    <div id="granted">
      <header>
        <h1>Welcome to Your Smart Home</h1>
        <p class="subtitle">Control your home with a single tap</p>
      </header>

      <div class="grid">
        <div class="card">
          <div class="card-header">
            <div class="card-icon"><i class="fas fa-home"></i></div>
            <h3 class="card-title">Home Status</h3>
          </div>
          <div class="status-item">
            <span class="status-label"><i class="fas fa-door-open"></i> Front Door</span>
            <span class="status-value" id="doorStatus"><span class="badge badge-danger">Unknown</span></span>
          </div>
          <div class="status-item">
            <span class="status-label"><i class="fas fa-window-maximize"></i> Living Room Window</span>
            <span class="status-value" id="windowStatus"><span class="badge badge-danger">Unknown</span></span>
          </div>
          <div class="status-item">
            <span class="status-label"><i class="fas fa-warehouse"></i> Garage</span>
            <span class="status-value" id="garageStatus"><span class="badge badge-danger">Unknown</span></span>
          </div>
          <div class="status-item">
            <span class="status-label"><i class="fas fa-thermometer-half"></i> Temperature</span>
            <span class="status-value" id="tempStatus"><span class="badge badge-success">Unknown</span></span>
          </div>
          <div class="status-item">
            <span class="status-label"><i class="fas fa-car"></i> Car Present</span>
            <span class="status-value" id="carStatus"><span class="badge badge-warning">Unknown</span></span>
          </div>
        </div>

        <div class="card">
          <div class="card-header">
            <div class="card-icon"><i class="fas fa-lightbulb"></i></div>
            <h3 class="card-title">Lighting Control</h3>
          </div>
          <div class="slider-container">
            <input type="range" min="0" max="255" value="" rawlital=" " + String(pwmValue) + R"rawlital(" class="slider" id="pwmSlider">
            <p>Brightness: <strong id="pwmDisplay">rawlital" + String(pwmValue) + R"rawlital(</strong></p>
          </div>
        </div>
      </div>

      <div class="grid">
        <div class="card">
          <div class="card-header">
            <div class="card-icon"><i class="fas fa-temperature-low"></i></div>
            <h3 class="card-title">Climate Control</h3>
          </div>
          <div class="temp-control">
            <div style="position: relative; flex: 1;">
              <input type="number" id="lowTemp" class="temp-input" placeholder="Low threshold" value="" rawlital" + String(tempLow) + R"rawlital(">
              <span style="position: absolute; right: 10px; top: 50%; transform: translateY(-50%); color: #6c757d;">°deg;C</span>
            </div>
            <div style="position: relative; flex: 1;">
              <input type="number" id="highTemp" class="temp-input" placeholder="High threshold" value="" rawlital" + String(tempHigh) + R"rawlital(">
              <span style="position: absolute; right: 10px; top: 50%; transform: translateY(-50%); color: #6c757d;">°deg;C</span>
            </div>
          </div>
          <div class="btn-group">
            <button class="btn btn-primary" onclick="setThresholds()"><i class="fas fa-save"></i> Save Settings</button>
          </div>
        </div>

        <div class="card">
          <div class="card-header">

```



```

    }
  });
}

// Slider control with immediate response
const slider = document.getElementById('pwmSlider');
const display = document.getElementById('pwmDisplay');
let lastSentValue = 0;
let updateTimeout;

slider.addEventListener('input', function() {
  const value = parseInt(this.value);
  display.textContent = value;

  if (Math.abs(value - lastSentValue) >= 5) {
    clearTimeout(updateTimeout);
    updateTimeout = setTimeout(() => {
      fetch('/light?value=' + value)
        .then(response => {
          if (response.ok) lastSentValue = value;
        });
    }, 30);
  }
});

function setPWM(val) {
  fetch('/light?value=' + val);
}

function setThresholds() {
  const low = document.getElementById('lowTemp').value;
  const high = document.getElementById('highTemp').value;
  fetch('/thresholds?low=' + low + '&high=' + high);
}

function closeGarage() {
  if (confirm("Are you sure you want to close the garage?")) {
    fetch('/closeGarage');
  }
}

```

```

        // Update UI every second
        setInterval(updateUI, 1000);
        updateUI(); // Initial update
    </script>
</body>
</html>
)rawliteral";

    server.send(200, "text/html", html);
}

void handleAuth() {
    if (server.hasArg("access")) {
        accessGranted = (server.arg("access") == "1");
        //accessGranted = false;
        preferences.putBool("access", accessGranted); // Reset on boot
    }
    server.send(200, "text/plain", "OK");
}

void handleUpdate() {
    if (server.hasArg("door")) doorStatus = server.arg("door");
    if (server.hasArg("window")) windowStatus = server.arg("window");
    if (server.hasArg("garage")) garageStatus = server.arg("garage");
    if (server.hasArg("temp")) temperature = server.arg("temp");
    if (server.hasArg("car")) carPresent = server.arg("car");
    server.send(200, "text/plain", "OK");
}

void handleToggleLock() {
    sendCommandToESP32("/cmd?toggleLock=1");
    server.send(200, "text/plain", "OK");
}

void handleToggleGarage() {
    sendCommandToESP32("/cmd?toggleGarage=1");
    server.send(200, "text/plain", "OK");
}

```

```

void handleLight() {
    if (server.hasArg("value")) {
        pwmValue = server.arg("value").toInt();
        preferences.putInt("pwm", pwmValue);
        sendCommandToESP32("/cmd?light=" + String(pwmValue));
    }
    server.send(200, "text/plain", "OK");
}

void handleThresholds() {
    if (server.hasArg("low") && server.hasArg("high")) {
        tempLow = server.arg("low").toInt();
        tempHigh = server.arg("high").toInt();
        preferences.putInt("tempLow", tempLow);
        preferences.putInt("tempHigh", tempHigh);
        sendCommandToESP32("/cmd?cool=" + String(tempHigh) + "&heat=" + String(tempLow));
    }
    server.send(200, "text/plain", "OK");
}

void handleCameraOn() {
    cameraActive = true;
    server.sendHeader("Location", "/stream");
    server.send(302);
}

void handleCameraOff() {
    cameraActive = false;
    server.sendHeader("Location", "/");
    server.send(302);
}

void handleStatus() {
    String json = "{";
    json += "\"door\": \"" + doorStatus + "\", ";
    json += "\"window\": \"" + windowStatus + "\", ";
    json += "\"garage\": \"" + garageStatus + "\", ";
    json += "\"temp\": \"" + temperature + "\", ";
    json += "\"car\": \"" + carPresent + "\", ";
    json += "\"pwmValue\": " + String(pwmValue) + ", ";
    json += "\"tempLow\": " + String(tempLow) + ", ";

```

```

    json += "\"tempHigh\": " + String(tempHigh) + ",";
    json += "\"accessGranted\": " + String(accessGranted ? "true" : "false");
    json += "}";
    server.send(200, "application/json; charset=utf-8", json);
}

void handleStream() {
    // First send the HTML page with the stream
    if (server.hasArg("stream") == false) {
        String html = R"rawliteral(
        <!DOCTYPE html><html><head>
        <title>Camera Stream</title>
        <meta name="viewport" content="width=device-width, initial-scale=1">
        <style>
            body { font-family: Arial, sans-serif; margin: 0; padding: 20px; background: #f5f5f5; }
            .container { max-width: 800px; margin: 0 auto; text-align: center; }
            .btn { background: #f44336; color: white; border: none; padding: 10px 15px; border-radius: 4px; cursor: pointer; margin: 20px; }
            img { max-width: 100%; height: auto; border-radius: 8px; margin-top: 20px; }
        </style>
        <script>
            function refreshImage() {
                const img = document.getElementById('stream');
                img.src = '/stream?stream=1&' + new Date().getTime();
            }

            // Refresh image every 100ms (10fps)
            setInterval(refreshImage, 100);
        </script>
        </head>
        <body>
            <div class="container">
                <h1>Camera Stream</h1>
                <button class="btn" onclick="window.location.href='/'">Back to Dashboard</button>
                <div>
                    
                </div>
            </div>
        </body>
        </html>
        )rawliteral";
    }
}

```



```

        }rawliteral";

        cameraActive = true;
        server.send(200, "text/html", html);
        return;
    }

    // Actual stream handling
    if (!cameraActive) {
        server.send(503, "text/plain", "Camera not active");
        return;
    }

    // Capture photo
    camera_fb_t *fb = esp_camera_fb_get();
    if (!fb) {
        Serial.println("Camera capture failed");
        server.send(500, "text/plain", "Camera capture failed");
        return;
    }

    // Send the photo
    server.setHeader("Cache-Control", "no-cache, no-store, must-revalidate");
    server.setHeader("Pragma", "no-cache");
    server.setHeader("Expires", "-1");
    server.setHeader("Access-Control-Allow-Origin", "*");
    server.send_P(200, "image/jpeg", (const char *)fb->buf, fb->len);

    // Return the frame buffer back to the driver for reuse
    esp_camera_fb_return(fb);
}

void sendCommandToESP32(const String& path) {
    HTTPClient http;
    http.begin("http://" + String(esp32Ip) + path);
    http.setTimeout(200); // Reduce timeout
    http.addHeader("Cache-Control", "no-cache");
    int httpCode = http.GET();
    if (httpCode != HTTP_CODE_OK) {
        Serial.printf("HTTP request failed, error: %s\n", http.errorToString(httpCode).c_str());
    }
    http.end();
}

```

10 Curriculum Vitae

PERSONAL INFORMATION



Cătălin Andrei GLIGOR

Strada Frunzisului, nr. 108, Cluj-Napoca

0755 485 281

catalinglor11@gmail.com

LinkedIn Profile: <https://www.linkedin.com/in/catalin-gligor-374871186>

Sex M | Date of birth 11/01/2003 | Nationality Romanian

WORK EXPERIENCE

July 2024 – August 2024

Full-stack Internship at TechQuarter

- Making a web application for the management of children in child day care centers, where the parent is the user; the parent can add a child to a specific day care center, check-in, check-out, check attendance, add allergies etc
- The application was done using C# and .NET for the backend part, SSMS for the database part and HTML, CSS, Tailwind, JavaScript/TypeScript for the frontend part

September 2024 - present time

Front End Developer Internship (Working student Program) at NTT Data

- Learned the basic/intermediate knowledge of a front end developer through a 10 month working student program
- Studied HTML, CSS, JavaScript/TypeScript and Angular through accredited Udemy courses and realised an individual project to showcase the acquired skills

EXTRACURRICULAR ACTIVITIES

November 2019, 2020

Participant in the Catalyst Coding Contest (CCC)

- Participated in an international programming competition designed to test and improve problem-solving and coding skills
- Worked in a team to solve a series of complex programming challenges within a limited timeframe, requiring advanced algorithmic thinking and teamwork
- In 2020, due to the pandemic, it was held online under the same circumstances

June 2019

Volunteer for Rosia Montana Marathon, Rosia Montana, Alba

- Guidance of participants and organizing various things during the marathon
- Worked as a team with other volunteers to ensure the good running of the event

EDUCATION AND TRAINING

October 2021 – present time

Technical University of Cluj-Napoca, Faculty of Electronics, Telecommunications and Information Technology (4th year student – currently)

- Main topics covered: Computer Programming (C/C++, Java, JavaScript), Microcontroller programming, Applied Informatics (Microsoft Office), Computer Aided Graphics (MATLAB), Microprocessors fundamentals and architecture – Assembly, Web Technologies.

September 2017– June 2021

National College "Avram Iancu", Câmpeni, MII, Computer Science Section

- Main topics covered: Computer Programming, Mathematics and English language.
- Baccalaureate Degree (Baccalaureate average grade: 9.26, Informatics grade: 10)

CERTIFICATES AND COURSES

June 2021

Web Programming Certificate - acquired in my last year of High School

October 2024

"The Complete Guide to HTML" - Udemy Course

November 2024

"Advanced CSS and Sass" - Udemy Course

November 2024

"GIT: Advanced commands" - Udemy Course

December 2024

"JavaScript - The Complete Guide 2025" - Udemy Course

January 2025

"TypeScript Crash Course" - Udemy Course

February 2025

"Angular - The Complete Guide (2025 edition)" - Udemy Course

PERSONAL SKILLS

Native language(s): Romanian

Other language(s):

	UNDERSTANDING		SPEAKING		WRITING
	Listening	Reading	Spoken interaction	Spoken production	
English	C1	C1	C1	C1	C1
French	A2	B1	A2	A2	A2

Levels: A1/2: Basic user - B1/2: Independent user - C1/2 Proficient user
Common European Framework of Reference for Languages

Communication skills:

- good communication skills gained by working on group projects in high school and university
- active listening skills
- increased sense of collaboration and teamwork

- Organizational skills:**
- good decision-making skills, taking initiatives during volunteering projects and organizing teamwork
 - good planning skills acquired while working on team projects
 - time management skills acquired while prioritizing exams and projects during college
- Job-related skills:**
- good knowledge and work experience with front-end technologies such as JavaScript/Typescript, Angular and React
 - good knowledge of back-end technologies, gained from university and personal projects, such as Java and Node.js
 - familiarity and experience with version control system such as Git
 - proficient in problem solving
 - immense drive, very dedicated to everything I do
 - passionate about technology and eager to learn new things and develop my skills in a hardworking team
 - advanced, almost fluent level of English, intermediate level of French
- Computer skills:**
- proficient in Microsoft Office™ Suite, including Word, Excel, PowerPoint
 - knowledge of a diverse range of programming languages and technologies: C/C++, PHP, JavaScript, HTML, CSS/SCSS, Python, C#, SQL, Java, ASM (x86) acquired when working on different personal projects and studied during high-school and university years
- Other skills:**
- good adaptability in different types of environments
 - fast learner, eager to develop both on a personal and professional level
 - understanding of the latest trends, standards, and developments regarding technology
- Driving licence:**
- B

ADDITIONAL INFORMATION

- Personal Projects:**
- Personal GitHub repository: <https://github.com/cati3000?tab=repositories>
 - Web Programming Certificate (September 2020-June 2021): Creating an online automotive parts shop using HTML, CSS, PHP and SQL.
 - Interactive Eye Tracker (November 2023 – January 2024): Made use of Python libraries such as OpenCV and dlib for facial and eye detection and gaze prediction. Developed algorithms for detecting the face, the eyes, isolating each eye and analysing and computing gaze position. Interactive part consists of different functionalities being used when looking in a certain direction, such as opening a browser, making a drawing using Turtle module, basic dice game made in Python terminal
 - Hotel Management App (May 2024): Developed an app using Java, HTML, CSS and JavaScript that searches for hotels stored in a JSON database, located within a specific radius; possibility to book a room and cancel it within 2 hours if needed
 - Habit Tracker App (June 2025): Habit tracking web application built with Node.js for the backend and React and Tailwind CSS for the frontend. Features user authentication, daily/weekly habit tracking, streak counting, and a dark/light mode toggle. Users can create habits, track their progress with an intuitive UI, and view statistics via a dashboard interface